UNIVERSITY OF CALIFORNIA
SANTA CRUZ

TECHNIQUES IN ACCELERATING DATA MANAGEMENT AND AI APPLICATIONS

A dissertation submitted in partial satisfaction of the
requirements for the degree of

Advancement Proposal for Doctor Of Philosophy

in

COMPUTER SCIENCE AND ENGINEERING

by

Jayjeet Chakraborty

August 2025

The Dissertation of Jayjeet Chakraborty
is approved:

Professor Chen Qian, Chair

Associate Professor Heiner Litz

Assistant Professor Andi Quinn

Professor Benedict Paten

Peter Biehl
Vice Provost and Dean of Graduate Studies

Copyright © by

Jayjeet Chakraborty

2025

Table of Contents

List of Figures **v**

List of Tables **vii**

1 Introduction **1**

2 Skyhook: Towards an Arrow-Native Storage System **5**
2.1 Introduction . . . 5
2.2 Background . . . 9
2.2.1 Ceph . . . 9
2.2.2 Apache Arrow . . . 10
2.2.3 Apache Parquet . . . 11
2.3 Design and Implementation . . . 11
2.3.1 Design Paradigm . . . 12
2.3.2 Skyhook: An Example Implementation . . . 12
2.3.3 Portability . . . 16
2.4 Evaluations . . . 19
2.4.1 Latency . . . 20
2.4.2 CPU Usage . . . 21
2.4.3 Network Traffic . . . 23
2.4.4 Crash Recovery . . . 23
2.5 Related Work . . . 24
2.6 Future Work . . . 26
2.7 Conclusion . . . 27

3 Thallus: An RDMA-based Columnar Data Transport Protocol **28**
3.1 Introduction . . . 28
3.2 Background . . . 30
3.2.1 Remote Procedure Call . . . 30
3.2.2 Remote Direct Memory Access . . . 31
3.2.3 Thallium Framework . . . 31
3.3 Serialization Overhead in Apache Arrow . . . 32

3.4 Thallus: Design and Implementation 32
3.5 Evaluation 37
3.6 Future Work 37
3.7 Conclusion 39

4 Characterizing the Performance of Approximate Nearest Neighbor Searches 41
4.1 Introduction 41
4.2 Background 43
4.2.1 LLMs and Retrieval Augmented Generation (RAG) 43
4.2.2 Vector Database and Approximate Nearest Neighbor (ANN) Search 44
4.2.3 Hierarchical Navigable Small World (HNSW) Algorithm 46
4.3 Performance Characteristics 48
4.3.1 Experiment Setup 49
4.3.2 HNSW Index Characteristics 51
4.3.3 CPU Hotspot Profiling 51
4.3.4 Top Down Micro-architecture Analysis 53
4.3.5 QPS / Throughput Scaling 54
4.3.6 Memory Bandwidth Scaling 55
4.4 Conclusion 59
4.5 Future Work 60

5 Future Project Ideas 62
5.1 How to support ANN searches on larger-than-DRAM datasets without compromising on performance ? 62
5.2 How to use heterogeneous compute hardware to accelerate an end-to-end AI application pipelines ? 64

6 Proposed Timeline 67

Bibliography 68

List of Figures

2.1 High-level design of Skyhook. 9
2.2 Architecture of Skyhook. 10
2.3 File layout design for Skyhook. 18
2.4 Query latency variation on scaling out from 4 to 8 and 16 storage nodes. 21
2.5 CPU utilization of scans without (left) and with (right) Skyhook. The thick blue line shows the CPU utilization on the client, and the thin lines show the CPU utilization on the storage servers. 22
2.6 Network bandwidth usage with and without Skyhook. 24
2.7 Network throughput as perceived from the client in the event of all OSDs crashing one by one. The steep drop in throughput is where all the OSDs crashed. 25

3.1 Control flow diagram showing the high-level operation of Thallus. 33
3.2 Comparison of data transport duration using Thallus versus Thallium RPC in column selectivity experiments. 38
3.3 Comparison of query execution duration using Thallus versus Thallium RPC in column selectivity experiments. 39

4.1 Architecture of a Retrieval Augmented Generation (RAG) system . . . 45
4.2 Structure of the Hierarchical Navigable Small World (HNSW) graph . . 48
4.3 Breakdown of the FAISS HNSW search function based on statistical profiling 53
4.4 QPS scaling with varying dataset size and query batch size 55
4.5 QPS scaling power law with dataset size and query batch size 56
4.6 STREAM triad bandwidth scaling of our test system with the number of cores 57
4.7 Memory bandwidth scaling with varying number of cores in FAISS HNSW search 58
4.8 Memory bandwidth scaling with varying dataset sizes in FAISS HNSW search 58

4.9 Memory bandwidth scaling with varying query batch sizes in FAISS HNSW search 59

List of Tables

4.1 File size and build duration of HNSW index with M=32 on varying slices of the Yandex Text-to-Image dataset 52
4.2 File size and build duration of HNSW index with varying M on a 1M slice of the Yandex Text-to-Image dataset 52
4.3 Top-Down Micro-architecture Analysis (TMA) of HNSW search 53

6.1 Timeline for completion 67

Chapter 1

Introduction

With the exponential advances in the capabilities and accuracy of Large Language Models (LLMs) and natural language understanding in the past 5 years, LLMs have found tremendous use cases in almost every sector of the industry. Industry and academia all over the world has been churning out bigger and better models every single day and LLMs are finding new use cases with every new improvement in its capabilities. In the history of computer science, no technology, be it the personal computer or the internet has grown at a pace similar to Generative AI.

As LLMs become more and more capable by the day, the pressure is on the data and compute infrastructure to be able to keep up with this growth. Training LLMs with 100's of billions of parameters requires storing, processing, and managing petabytes of data scraped from the internet. This massive amounts of data are ingested into cloud object stores such as Amazon S3 by the internet scrapers. Not all of the data from the internet has useful context to train an LLM, hence several iterations of

data cleaning, pre-processing, and filtration is required to make the data suitable for training LLMs with high accuracy and less hallucinations. Processing these massive datasets is traditionally done by deploying large clusters of distributed data processing systems such as Apache Spark and Hadoop, and then reading the data into these clusters for processing and then writing them to back to the object stores. Moving these huge amounts of data over the wire to the data processing clusters is highly inefficient, requires a lot of energy, and greatly slows down the entire data cleaning and pre-processing stages. Additionally, these data processing clusters usually have 10s of physical nodes and several 100s of CPU cores. Whereas, the machines backing the cloud object storage systems are in the order of ten's of thousands and have hundreds of thousands of cores. But unfortunately these cores usually remain highly underutilized because of the I/O heavy nature of cloud object store workloads.

With Skyhook, our programmable object storage based on Ceph, we explore how common data processing workloads could be offloaded to the object storage servers in distributed object storage systems such as Ceph. We show that by offloading data processing tasks such as filters and projections to the object storage servers, it is possible to mitigate the bottleneck of limited CPU resources in the data processing cluster's compute servers, and gain-back massive scalability. Additionally, processing data directly inside the storage, essentially moving compute near to the data, mitigates the need to transport data over the wire and reduces the end-to-end data and query processing latency.

While experimenting with Skyhook and measuring the performance acceler-

ation realized by offloading data processing tasks to the storage layer, we found that processing columnar data near storage requires deserialization of the data from its compressed binary format, processing of it, re-serialization of the results into a TCP/IP conforming contiguous format, and deserialization again in data processing clusters nodes. This extra serialization and deserialization of data has high overhead and reduces the gains that could be realized from Skyhook. We also found that not only in programmable storage systems like Skyhook, but in any data processing system where columnar data is exchanged between servers, serialization of in-memory columnar data into a contiguous format as required by TCP/IP based protocols has a high overhead and needs to be hardware accelerated. With Thallus, our RDMA-based columnar data transport protocol, we explore using modern networking technologies such as RDMA with Infiniband to transport columnar data between servers efficiently without requiring serialization, expensive memory copies, or any CPU involvement. Our results show that using scatter-gather DMA engines through RDMA/Infiniband provides speedup over traditional TCP/IP based RPC protocols when transporting columnar data.

With LLMs finding use cases in a large number of industries, people have tried to make LLMs smarter in a particular domain besides it generic knowledge. Two common ways of making LLMs domain-specific is through finetuning and Retrieval Augmented Generation (RAG). Finetuning involves taking an out-of-the-box pre-trained model and re-training it over domain specific data. This adjusts the weights of the model to make it better at generating answers from that specific domain rather than the entire internet data. Finetuning often generates re-trained LLMs that exhibit high

accuracy but it comes at a huge cost of infrastructure, energy, and time. Also, the finetuned LLMs loses it generic capabilities, greatly diminishing the benefits of finetuning altogether. RAG on the other hand does not require any expensive finetuning step but rather leverages the context ingestion capabilities of LLMs to generate answers on custom context. In a RAG system, domain specific data is tokenized and stored in a vector database which allows searching related context from a large pool of information based on a user query. This context is sent to the out-of-the-box LLM along with the query to generate the answers. As can be understood, vector databases sit on the critical path of a RAG system and contributes a large segment of the query-to-answer latency. Approximate Nearest Neighbor (ANN) algorithms, which form the heart of vector databases need to be highly efficient at searching related context and is a critical candidate for hardware acceleration. Accelerating any workload with modern hardware requires in-depth understanding of its performance characteristics. We take the HNSW algorithm, a widely-used graph-based ANN search algorithm and study it performance characteristics in terms of throughput, scalability, compute and memory resource utilization, potential sources of bottleneck among others. We believe that such a detailed performance characterization will help guide researchers while developing hardware and software strategies for accelerating this workload and in-turn the end-to-end RAG pipeline.

Chapter 2

Skyhook: Towards an Arrow-Native Storage System

2.1 Introduction

Over the last two decades, a variety of distributed data processing systems such as Spark [117], Ray [74], and Dask [88] have come into existence. These systems were built to efficiently query vast quantities of structured and unstructured data and get insights fast. In the current era of AI, performance of these systems have become more critical than ever before, as they sit on the critical path of data flow in the broader LLM training and real-time inference systems (e.g. RAG). With continued improvement in the performance of LLM training and inference systems, it is critical that data is fed into this systems at a pace that does not slow down the entire AI pipeline. This requires continuous improvement in performance of these distributed data processing

systems. Unlike traditional relational database management systems (RDBMS) such as PostgreSQL [11], which are designed to optimally manage both data storage and processing, these systems were designed to read data from a wide variety of data sources, including those in the cloud like AWS S3 [26]. These systems depend on different file formats like Parquet [1], Lance [81], and ORC [18] for efficiently storing and accessing data. Since storage devices have been the main bottleneck in data processing systems, the primary focus of these file formats has been to store data efficiently in a compressed and encoded format and reduce the amount of disk I/O required to fetch the data. However, with recent advancements in storage devices with PCIe Gen5 NVMe [115] drives and NICs with Infiniband support [84], the bottleneck has shifted from the storage devices to the client machine's CPUs, making the notion of "A fast CPU and slow disk" invalid, as shown by Trivedi et al. [98]. The serialized and compressed nature of these file formats makes reading from them CPU-limited in systems with high-speed network and storage devices, resulting in severely affected scalability.

An attractive solution to this problem is to offload as much computation as possible to the storage layer to achieve scalability and reduced data movement, leading to faster end-to-end query processing. Several popular distributed data processing systems have explored this approach, e.g. IBM Netazza [94], Oracle Exadata [80], Redshift [56], and PolarDB [36]. Most of these systems are built following a clean-slate approach and use specialized and expensive hardware, such as Samsung's SmartSSDs [49] and FPGAs [75] with custom logic for table scanning. Building vertically-integrated data management systems like these require in-depth understanding and expertise in these

custom hardware blocks and are not suited for widespread adoption. Also, modifying existing systems like MariaDB [87], as in the case of YourSQL [65], requires modifying legacy code that is optimized over the years, which might result in performance, security, and reliability issues. A possible way to avoid these issues is to have programmable storage systems with low-level extension mechanisms that allow implementing application-specific data manipulation and access in their I/O path. Customizing storage systems via plugins results in minimal implementation overhead and increases the maintainability of the software.

Programmable object storage systems such as Ceph [106, 111], Swift [78], and DAOS [70] often provide a POSIX filesystem interface for reading and writing files which are mostly built on top of object storage access libraries such as "`librados`" in Ceph and "`libdaos`" in DAOS. Being programmable, these systems provide plugin-based extension mechanisms that allow direct access and manipulation of objects within the storage layer. We leverage these features of programmable storage systems and develop a new design paradigm that allows embedding widely-used data access libraries inside the storage layer. As shown in Figure 2.1, the extensions on the client and storage layers allow an application to execute data access library operations either on the client or via the direct object access layer inside the storage servers.

We implement Skyhook, an instantiation of our design paradigm using RADOS [108] as the programmable object storage backend, CephFS [33] as the POSIX layer, Apache Arrow [97] as the data access library, and Parquet as the file format. We evaluate the performance of Skyhook by scaling out the cluster measuring metrics such

as query latency, CPU utilization, and network traffic. The evaluations show that Skyhook scales by offloading CPU usage for common data processing tasks to the storage layer, freeing the client for other processing tasks. Additionally, since compute units are co-located with storage nodes in Skyhook, the crash recovery and consistency semantics of the storage layer apply naturally to the query processing layer, and Skyhook queries become fault-tolerant. Since Skyhook uses Arrow extensively, we contributed our source code to the upstream Apache Arrow open-source project [15] [14].

In summary, our primary contributions are as follows:

- A design that embeds widely-used data access libraries inside storage servers using a plugin infrastructure without requiring any changes to the data access library and has no impact on the storage system's resilience and failure management.
- A design that extends a widely-used data access library with a plugin to allow offloading tasks from clients to storage servers by leveraging existing and unmodified filesystem and object storage interfaces (assuming an extensible object storage system).
- A brief analysis of the performance gain achieved by offloading dataset scans to the storage servers. We demonstrate that offloading dataset scan operations to the storage layer results in faster queries, enhanced scalability, and reduced network traffic.

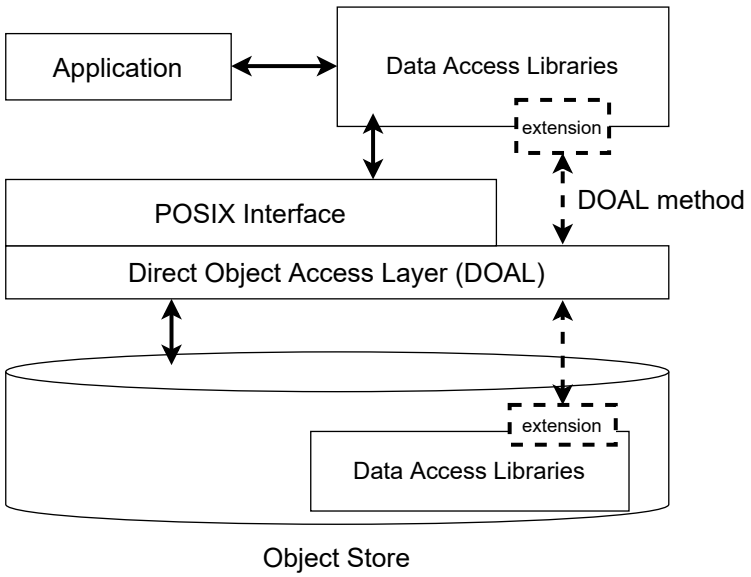


Figure 2.1: High-level design of Skyhook.

2.2 Background

2.2.1 Ceph

Ceph is an open-source software-defined storage platform that implements a distributed object storage layer and provides 3-in-1 interfaces for object, block, and file-level storage. One of the main properties of Ceph is that it does not have a single point of failure because of its CRUSH [107] map feature. On a high-level, a CRUSH map contains object-OSD mappings, which are downloaded to the client when connecting to a Ceph cluster. The client uses the mapping information to calculate the location of an object in the cluster and accesses the object directly. The object storage layer of Ceph, RADOS, is programmable as Ceph provides a plugin-based extension mechanism with its Object Class SDK [4] that allows extending the object storage layer with custom

plugins written in languages such as C++ and Lua. These plugins are embedded in the form of shared libraries inside Ceph OSDs and provide the ability to access and manipulate RADOS objects on the fly within the RADOS I/O path. The Object Class SDK offers a small subset of POSIX APIs such as `read`, `write`, and `stat`. Several Ceph components, such as the RADOS Gateway (RGW) and the RADOS Block Device (RBD), use the Ceph Object Class SDK. We refer to this SDK as the Ceph CLS in the rest of the paper.

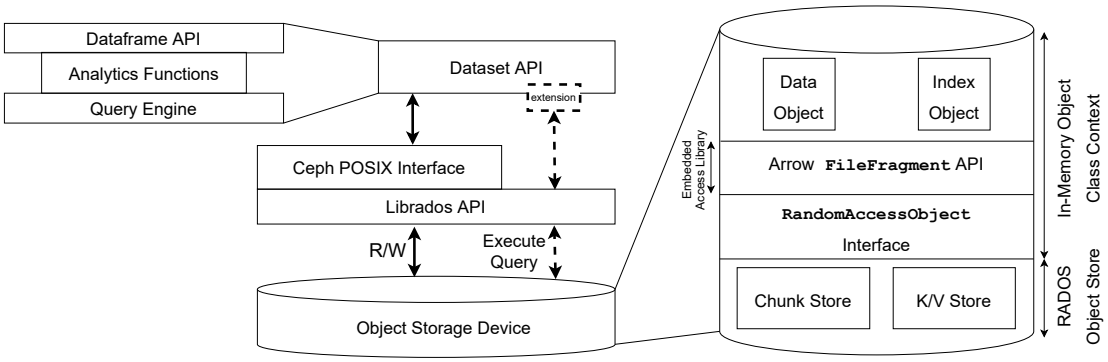


Figure 2.2: Architecture of Skyhook.

2.2.2 Apache Arrow

Arrow is a zero-copy columnar in-memory format for storing structured data optimized for efficient analytics operations on modern hardware. Arrow's uniform disk, memory, and wire format allow applications to overcome any serialization and deserialization overhead while transferring Arrow data between different processes and systems. Apart from being a highly-efficient in-memory format, Arrow is also a collection of several modular data processing components, collectively utilizing which allows building

parts or all of a data processing system. Some of the data-processing pieces that Arrow provides are as follows: Gandiva, A LLVM-based expression compiler [19]; Flight, A gRPc-based data transport interface [20]; Dataset API, A datasets abstraction [2]; Arrow IPC, A binary data transfer protocol. Popular data processing systems such as Spark, Dask [88], and Ray [74] support the Arrow in-memory format.

2.2.3 Apache Parquet

Parquet is an open-source columnar file format for efficient and long-term storage of tabular data. Parquet supports several encoding schemes such as Dictionary and Run-Length Encoding (RLE) and compression schemes such as Snappy, LZ4, and ZSTD, which makes it highly space-efficient. One of the essential features of Parquet is its ability to push down predicates. Parquet files contain row group statistics and column offsets in their footer metadata, allowing Parquet file readers to skip over row groups and column data pages containing non-relevant data. In this way, using Parquet for data storage greatly minimizes the I/O required for reads. These characteristics of Parquet make it the de-facto choice for most data processing systems in the Hadoop ecosystem.

2.3 Design and Implementation

In this section, we discuss our design paradigm, the motivation behind it and provide an in-depth description of the internals of Skyhook. Additionally, we discuss the portability of our design paradigm to other storage systems and data access libraries.

2.3.1 Design Paradigm

One of the most critical aspects of our design is that it allows building in-storage data processing systems with minimal implementation effort. Our design extends the client and storage layers of programmable object storage systems with widely-used data access libraries requiring absolutely no modifications. We achieve this by (1) creating a filesystem shim in the object storage layer so that data access libraries embedded in the storage layer can continue to operate on files, (2) mapping client requests-to-be-offloaded directly to objects using filesystem striping metadata, and (3) mapping files to logically self-contained fragments by using standard filesystem striping. As shown in figure 2.2, we developed one instantiation of our design paradigm using Ceph as the storage system, Apache Arrow as the data access library, and Parquet as the file format. We expose our implementation called Skyhook via the Arrow Dataset API by creating a new file format abstraction named the `SkyhookFileFormat`, that extends the `FileFormat` API [3] in Arrow. In the next section, we discuss the internals of Skyhook in detail.

2.3.2 Skyhook: An Example Implementation

Extending the Ceph Object Store. In the storage layer, using the Ceph CLS, we create a "`scan_op`" object class method that reads a RADOS object containing tabular data encoded in binary file formats such as Parquet and ORC, scans it using Arrow APIs by applying the query parameters received from the client and returns the scanned result in the form of an Arrow table. Reading from binary files requires a

filesystem shim (a filesystem-like random access interface) to seek around in a file and read from any offset. Since the Ceph CLS does not provide a full POSIX-like API, for example, with a "`seek`" method, we utilize the Ceph CLS APIs to create a filesystem shim that allows seeks by keeping track of the file pointer, thereby providing a file-like view over objects. Arrow provides a `FileFragment` API that wraps a file and allows applying scan operations to it. It takes predicates and projections as inputs, applies them to a file, and returns the result in the form of an Arrow table. Since the filesystem shim allows interacting with objects as files, it seamlessly plugs into the `FileFragment` API.

Tunneling through CephFS. In Skyhook, files are written to Ceph using the CephFS POSIX interface, which stripes files over fixed-size objects stored in the RADOS layer. During the read phase, to execute the "`scan_op`" Ceph CLS method on a RADOS object, Skyhook must first map a file to a RADOS object. This mapping is achieved by leveraging the file striping metadata provided by the Metadata Server (MDS) of CephFS to calculate object IDs from file names. We implement a direct object access (DOA) API on the client-side that facilitates this translation and invokes the Ceph CLS method "`scan_op`" on the corresponding RADOS object. The DOA API takes as input a serialized flatbuffer [55] which comprises the scan request and contains the parameters as shown in Listing 1. The "`scan_op`" method is invoked with the serialized scan request as input. Using the DOA API, the client can interact with RADOS objects and manipulate them directly in application-specific ways while also having a filesystem view over the objects. It is almost impossible to implement reads

```
table ScanRequest {
  file_size: long;
  file_format: short;
  filter_expression: [ubyte];
  partition_expression: [ubyte];
  dataset_schema: [ubyte];
  projection_schema: [ubyte];
}
```

Listing 1: Scan request in Skyhook.

without the DOA API because the dynamically generated result data inside the storage servers is not byte-addressable, implying a filesystem interface cannot be used.

Extending the Arrow Dataset API. The Arrow framework provides a `FileFormat` API that plugs into the Dataset API [2] and allows scanning datasets in a unified manner. It provides APIs for scanning datasets of different file formats such as Parquet, ORC, CSV, and Feather [110]. Since Parquet is the de-facto file format in most modern data processing systems, we use it as a baseline for our performance evaluations. We extend the `FileFormat` API in Arrow to create a `SkyhookFileFormat` API that leverages the DOA API to enable offloading fragment scan operations to Ceph OSDs. The `SkyhookFileFormat` API serializes the query parameters and passes them on to the DOA API for sending them to the storage layer. This API is generic enough to offload scan operations on every file format supported by Arrow as long as there is

a Ceph CLS method to scan the particular file format in the storage layer. This API allows client applications using the Arrow Dataset API to offload file scan operations to the RADOS layer in Ceph by simply changing the file format argument in the Dataset API as shown in Listing 2.

Establishing a File Layout Design. Being very efficient in storing and accessing data, Parquet has become the de-facto file format for popular data processing systems like Spark and Hadoop. Since Parquet files are often multiple gigabytes in size, a standard way to store Parquet files is to store them in blocks as in HDFS [34, 93], where the typical block size is 128 MB [5]. While writing Parquet files to HDFS, each row group is stored in a single block to prevent reading across multiple blocks when accessing a single row group. We aim to follow a similar file layout for storing Parquet files in Ceph so that every row group is self-contained within an object. In our design, a Parquet file with R row groups is split into R Parquet files, each containing the data and metadata from a single row group. To retain the optimizations due to the predicate pushdown capability in Parquet, the footer metadata and the schema of the parent Parquet file are also serialized and written to a separate Parquet file ending with a ".index" extension. So, for every Parquet file containing R row groups, $R + 1$ small Parquet files are written, each contained within a single RADOS object. We refer to this set of $R + 1$ Parquet files as a logical Parquet file. During the dataset discovery phase, we discover only those files that end with a ".index" extension and read out the schema information from them. During the query execution phase, for every logical Parquet file, first, the footer metadata from the index file is read. Then, the IDs of the row groups

that qualify for scanning are calculated based on the row group statistics present in them. Finally, the row group IDs are translated to the corresponding filenames, and then the underlying objects of these files are scanned in parallel via the DOA API. Similar to scanning Parquet files containing data from row groups, the index Parquet file is also scanned using a Ceph CLS method. The index file feature discussed here is only a design proposal and is yet to be implemented in Skyhook. Figure 2.3 shows the file layout design.

2.3.3 Portability

The implementation of Skyhook consists of several pluggable and reusable components that make it feasible to port Skyhook's design to other systems. In this section, we discuss how the design paradigm of Skyhook can be applied to other programmable object storage systems and data-access libraries by specifying the requirements in both the client and the storage layers.

We start by discussing the portability of Skyhook's design to other programmable object storage systems. The DOA and the storage connection APIs defined in the `SkyhookFileFormat` API on the client-side needs to use the object storage system specific access library to be able to connect to the storage layer and execute object class methods in the storage nodes. On the storage-side, the filesystem shim defined in the storage plugin needs to use the storage system specific object class SDK to access the corresponding object store. Additionally, the storage system should have a POSIX interface that allows using a user-defined stripe unit while writing files. This is essential

```
# Reading from Parquet
import pyarrow.dataset as ds
format_ = "parquet"
dataset = ds.dataset(
    "/dataset", format=format_
)
dataset.to_table()


# Reading from Parquet using Skyhook
import pyarrow.dataset as ds
format_ = ds.SkyhookFileFormat(
    "parquet", "/ceph.conf"
)
dataset = ds.dataset(
    "/dataset", format=format_
)
dataset.to_table()
```

Listing 2: Reading from a Parquet dataset **with and without** Skyhook using the Arrow Dataset API.

to ensure that every file is backed by a single object.

Skyhook's design paradigm can be ported to other data access libraries. To achieve this, the data access library on the client-side should have a Dataset API to discover and instantiate a dataset abstraction over a directory of files. It should be able to map file scan calls to object class method calls using the DOA API. Additionally, a format-specific file writer is required to split large files into object-sized ones and write them in the POSIX filesystem by manipulating the stripe unit. In the storage nodes, the data access library should be embeddable in the form of shared libraries and should compile with the storage plugin. Most importantly, it needs to use the filesystem shim as a data source to read from objects as files.

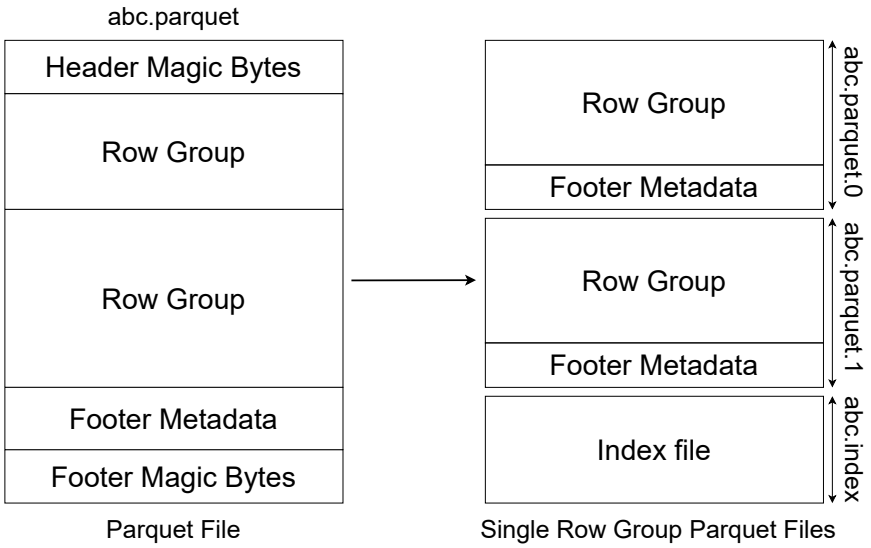


Figure 2.3: File layout design for Skyhook.

2.4 Evaluations

We performed experiments to compare the query latency, CPU utilization, and network bandwidth usage of accessing a dataset and filtering at the client or the storage server. The experiments were performed on CloudLab, the NSF-funded bare-metal-as-a-service infrastructure [52]. For our experiments, we exclusively used machines with an 8-core Intel Xeon D-1548 2.0 GHz processor (with hyperthreading enabled), 64 GB DRAM, a 256 GB NVMe drive, and a 10 GbE network interface. These bare-metal nodes are codenamed "m510" in CloudLab. We ran our experiments on Ceph clusters with 4, 8, and 16 storage nodes and a single client node. Each storage node had a single Ceph OSD running on the NVMe drive. We configured the OSDs to use 8 threads to prevent contention due to hyperthreading in the storage nodes. The number of placement groups (PGs) was increased from 128 to 512 while scaling out the cluster to avoid increased lock contention. A CephFS interface was created on a 3-way replicated pool and was mounted in user mode using the ceph-fuse utility.

We used a dataset with 1.2 billion rows and 17 columns consisting of data from the NYC yellow taxi dataset [7] as our workload. The in-memory size of the dataset was found to be about 155 GB. In this work, we experimented with 64 MB files as we discovered that 64 MB files produced the best results for both with and without Skyhook cases. The dataset was generated by replicating a 64 MB Parquet file 460 times. The Parquet files used were uncompressed. The fact that reading directly from RADOS bypasses any cache allowed us to replicate the same file without any caching-related

implications on performance. Since a row group is supposed to be self-contained within a single object and the unit of parallelism in the Arrow Dataset API when using Parquet is a single row group, we used Parquet files having a single row group backed by a single RADOS object in all our experiments. Accordingly, the underlying object size was also 64 MB. Since the index file feature was not available in Skyhook, we used similarly configured datasets containing only single row group 64 MB Parquet files for both with and without Skyhook experiments. We used the Python version of the Arrow Dataset API for the experiments and used Python's ThreadPoolExecutor for launching scans in parallel, following an asynchronous I/O model. We measured latency, scalability, CPU usage, and network traffic of scanning a Parquet dataset with and without Skyhook.

2.4.1 Latency

We ran queries to select 100%, 99%, 75%, 50%, 25%, 10%, and 1% of the rows from our dataset. The queries were crafted so that the rows always get selected from the beginning of the row group in every file. So, 10% of the rows indicates that the first 10% of the rows were selected from every file. In the 100% selectivity case, Skyhook reads and returns all the rows without applying any filters to the dataset. The IO depth at each storage node was maintained at 2 across all the experiments. As shown in Figure 2.4, scanning with Skyhook is slower than without Skyhook in the 100% scenario since there is no reduction of data sent over the network, and on top of that, Skyhook ends up sending more data as it uses LZ4-compressed Arrow IPC format on the wire which has a slightly larger byte size than the uncompressed Parquet format. On scaling out from

4 to 8 nodes, the performance of scans, both with and without Skyhook, improved as there were still CPU resources available to use. But, on scaling out from 8 to 16 nodes, the scan without Skyhook bottlenecked on the client CPU usage and stopped scaling further, whereas the scans with Skyhook, being not bottlenecked, scaled out. Skyhook maintains scalability as it can offload and distribute computation across all the storage nodes rather than staying CPU bottlenecked on the client. Also, the conversion from in-memory to wire format of Arrow requires multiple small memory copies whose overhead add up to cause significant performance degradation in Skyhook. This performance degradation is visible in the 4 nodes scenario because the without Skyhook scan is not bottlenecked, and there is nothing to trade-off the conversion overhead of Skyhook scans as in the 8 and 16 nodes cases.

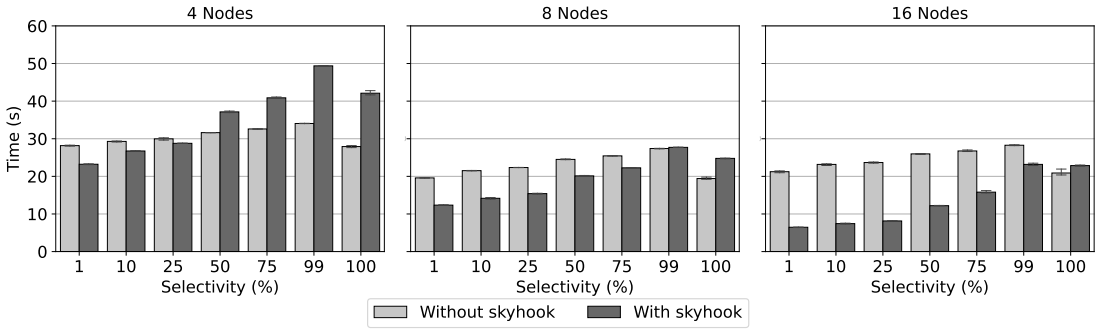


Figure 2.4: Query latency variation on scaling out from 4 to 8 and 16 storage nodes.

2.4.2 CPU Usage

Figure 2.5 shows the CPU utilization over time by the client and the storage layers during a query execution with 100% selectivity on a cluster with 16 storage nodes.

The CPU usage has been recorded using Prometheus's [12] node metrics exporter. We observe that Parquet almost exhausts the client's CPU even with no filtering involved. This observation implies that the client would be unable to do any other processing work, and the query performance is bottlenecked on the client's CPU. On the other hand, we observe that with Skyhook, plenty of CPU is available on the client node, and there is high CPU utilization on the storage layer. The CPU usage on the client node can be attributed to the decompression of LZ4-compressed Arrow IPC data received as the result of scan operations. Hence, with plenty of CPU available on the client-side, more asynchronous threads can be launched to improve parallelism, or the client can take other processing tasks.

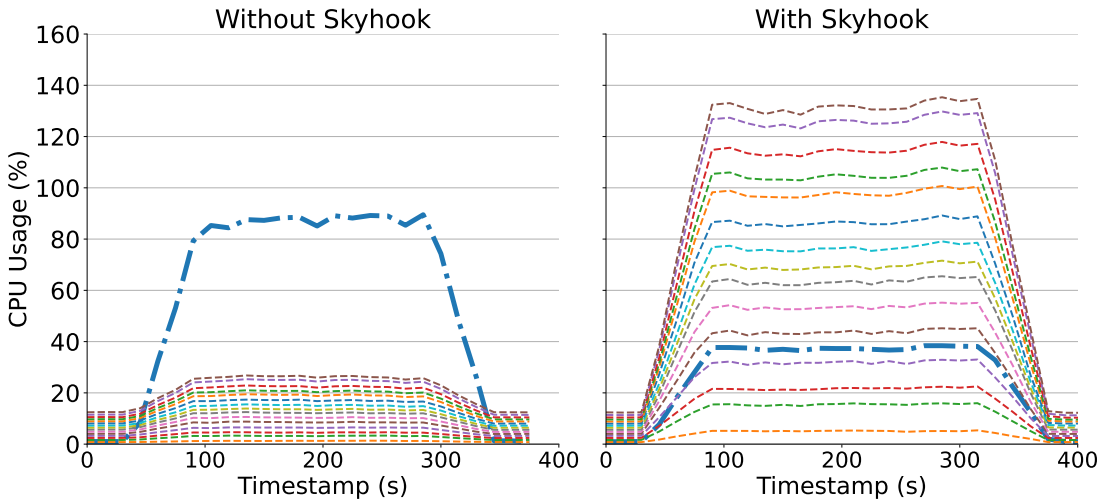


Figure 2.5: CPU utilization of scans without (left) and with (right) Skyhook. The thick blue line shows the CPU utilization on the client, and the thin lines show the CPU utilization on the storage servers.

2.4.3 Network Traffic

While performing our experiments, we recorded the network traffic received on the client node. Figure 2.6 shows the network throughput for query executions with 100%, 10%, and 1% selectivities on a cluster with 16 storage nodes. On scanning with Skyhook, the network traffic keeps reducing as we move from lower (100%) to higher (1%) selectivities. In contrast, without Skyhook, the network traffic always remains the same, which implies significant bandwidth wastage. The numbers recorded show that using Skyhook results in significant data movement reduction. The network traffic for Skyhook is slightly higher than without Skyhook for the 100% scenario because the LZ4-compressed Arrow IPC format that Skyhook uses for transferring data is slightly larger in byte size than the uncompressed Parquet binary format, and moving that extra data becomes an overhead since, during 100% selectivity, all of the data is sent back in both cases.

2.4.4 Crash Recovery

To verify the crash recovery functionality while using Skyhook, we executed a long-running query with 100% row selectivity and restarted all the OSDs one-by-one in about the middle of the query execution to simulate a crash. As shown in Figure 2.7, the query execution throughput resumed back to what was before the crash as the Ceph OSDs started restarting. This result shows that Skyhook queries are naturally fault-tolerant due to the co-location of query execution units with fault-tolerant storage nodes.

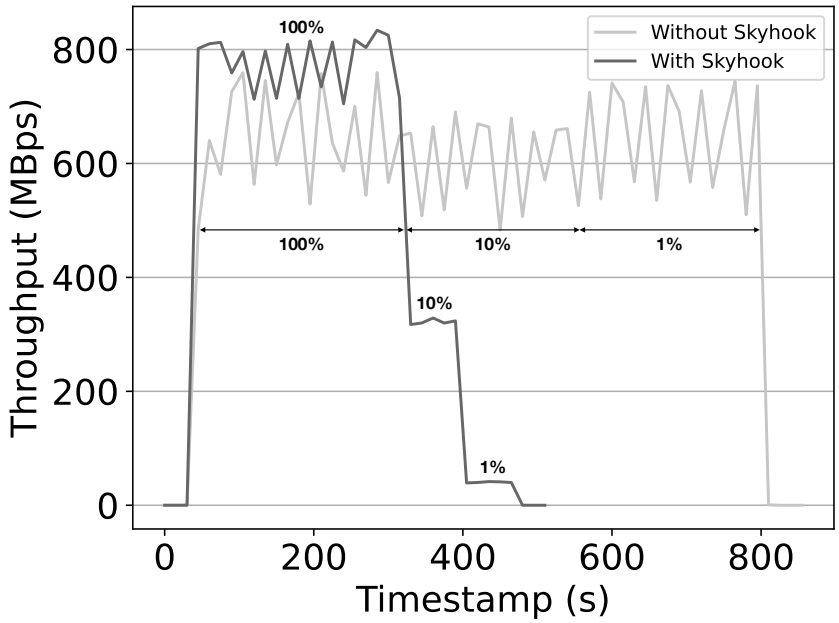


Figure 2.6: Network bandwidth usage with and without Skyhook.

2.5 Related Work

Several distributed data processing systems have embraced the idea of query offloading to the storage layer for performance improvement. The paper on the Albis [98] file format by Trivedi et al. explored that with high-performance storage and networks, CPUs have become the new bottleneck. Since the CPU bottleneck on the client hampers scalability, offloading CPU to the storage layer has become more important. Recently, S3 introduced S3 Select [60], which allows files in either Parquet, CSV, or JSON format to be scanned inside S3 for improved query performance. Many other systems which already support reading from S3, e.g. Spark [118], Redshift [56], Snowflake [46], and PushdownDB [116] have started taking advantage of S3 Select. But being an IaaS [73], the performance of S3 Select cannot be tuned, nor can it be customized to read from file

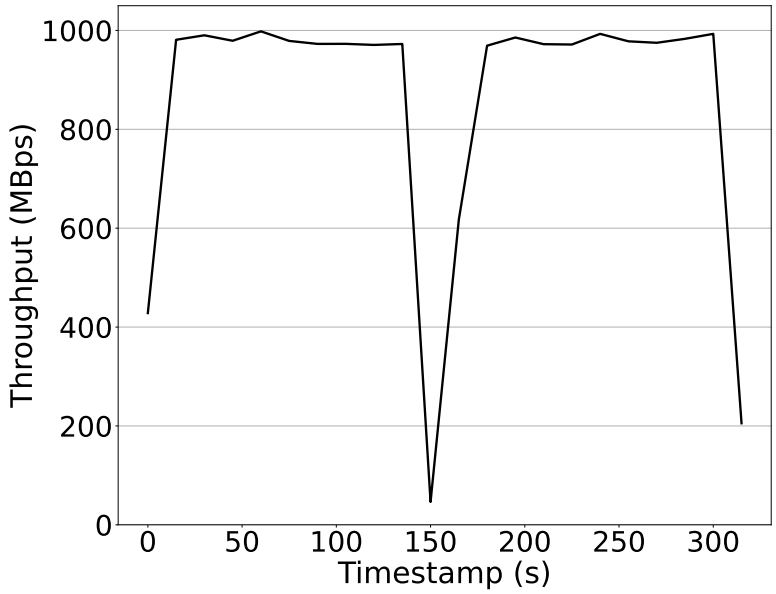


Figure 2.7: Network throughput as perceived from the client in the event of all OSDs crashing one by one. The steep drop in throughput is where all the OSDs crashed.

formats except the ones it supports. Systems like IBM Netezza [94], PolarDB [36], and Ibex [112] depend on sophisticated and costly hardware like FPGAs and Smart SSDs to perform table scanning inside the storage layer. These systems employ hardware-software co-design techniques to serve their specific use cases. These systems generally follow a clean-slate approach and are built from the ground up specifically tailored for query offloading. The recent work by Adams et al. [24] shows that near-data processing can be enabled without requiring modifications by ensuring that data elements do not cross shard boundaries and by sharing data layout information with the clients. On a high level, this approach is somewhat similar to that of Skyhook, except that the work is in its early stages, and the authors do not provide any performance evaluation of their prototype implementation.

In our approach, we take a programmable storage system, Ceph, and extend its filesystem and object storage layers to allow offloading queries leveraging the extension mechanisms it provides. Storage systems like OpenStack Swift [78] and DAOS [70] also provide extension mechanisms via Storelets [79] and DAOS middleware [62] respectively. We embed Arrow libraries inside the storage layer to build the data access logic. Our approach signifies that storage systems should provide plugin mechanisms so that they can be easily extended to support ad-hoc functionality and do not need modification of legacy code or require a complete rebuild.

2.6 Future Work

As discussed in Section 2.4.1, currently, Skyhook's performance suffers from the in-memory to wire serialization overhead of Apache Arrow. We believe that employing techniques such as using RDMA for transferring data over the network can help alleviate this serialization overhead. Implementing this would require changes in the wire protocols of both Arrow and Ceph. On the use cases front, we are currently working on integrating Skyhook with in-memory SQL-based query engines such as DuckDB [86] and evaluating the performance over industry-standard benchmarks such as TPC-H and TPC-DS. We also plan on assessing the performance of Skyhook with modern distributed computing systems such as Dask and Ray, as their recent addition of Dataset API support has made their integration with Skyhook seamless. Another line of future work deals with using Gandiva to accelerate the query processing performance of Arrow

inside the storage layer. Finally, we plan on implementing a Flight-based communication protocol in Ceph to make the system faster and more arrow-native.

2.7 Conclusion

This paper presents a new design paradigm that allows extending the POSIX interface and the object storage layer of programmable object storage systems with plugins to allow offloading compute-heavy data processing tasks to the storage layer. We discuss how to embed different data access libraries and processing frameworks inside plugins to build a universal data processing engine that supports various file formats. We present an instantiation of this design, Skyhook, implemented using Ceph for the storage system and Arrow for the data processing layer. Currently, Skyhook supports reading Parquet files only, but we can easily add support for other file formats since we use Arrow as our data access library. We also discuss our file layout design for storing Parquet files in Ceph, similar to HDFS that allows efficient querying. We expose our implementation via a `SkyhookFileFormat` API, an extension of the Arrow `FileFormat` API, and also contributed our code to the Apache Arrow open-source project. Additionally, we present a brief performance evaluation of Skyhook and demonstrate that offloading compute-heavy query execution to the storage layer helps improve query performance by reducing data movement significantly.

Chapter 3

Thallus: An RDMA-based Columnar Data Transport Protocol

3.1 Introduction

The past decade has seen a surge in data processing tools and technologies due to the exponential growth of data from IoT devices, sensors, social networks, and financial institutions. The demand for efficient data processing systems has surged in the past couple years due to the exponential growth in Generative AI workloads. Modern data processing systems such as Dask, Spark, and BigQuery exchange vast amounts of data over the internal data center network during query operations, especially in the re-partition and shuffle stages of the query plan. Since these data exchanges are critical to the progress of the query processing workloads, the network stack involved in these systems must exhibit optimal performance.

Unfortunately, these data exchanges suffer from serialization overhead, where the in-memory data needs to be serialized into a contiguous representation for the data transport framework to be able to transport it over the wire. This is even the case for columnar in-memory formats such as Apache Arrow, which, although zero-copy within the same host, requires expensive serialization when transferring between hosts. This serialization requirement arises from using TCP/IP-based data transport frameworks since using TCP/IP requires the data to be contiguous before feeding into the NIC. In the past decade, I/O has become faster, with disk I/O reaching bandwidths of 10-15GB/s with PCIe Gen5 NVMe SSDs and network bandwidth reaching up to 900 Gbps with technologies such as NVLink [104]. With such advancements in the performance of the I/O stack, data serialization has become a significant source of overhead in network-bound data center applications.

One approach to solving this problem is using RDMA (Remote Direct Memory Access) to transport columnar data buffers between the machines' memories without any CPU involvement. Although there has been previous work studying the use of RDMA for accelerating network applications and columnar databases, most of it is either based on research prototypes or used internally in proprietary systems (e.g. PolarDB [36]), and none is open-sourced. To our knowledge, no work has been done to explore transporting widely-used open-source in-memory columnar data formats such as Apache Arrow [97] over a RDMA-based network stack.

This work studies the serialization overhead incurred when transporting Apache Arrow data over a TCP/IP-based RPC interface implemented using the Thallium frame-

work from the Mochi [89] ecosystem. We choose Apache Arrow because it is one of the most widely used columnar in-memory data formats today, and almost all major distributed data processing systems have either adopted it or are working actively to support it. We design and implement Thallus, a data transport protocol based on RDMA to transport Apache Arrow data, leveraging the Thallium framework. We demonstrate that using Thallus to transport columnar data results in up to 5.5x data transport performance improvement and up to 2.5x faster query execution over plain Thallium RPCs.

3.2 Background

3.2.1 Remote Procedure Call

Remote Procedure Call (RPC) is a protocol that allows a process on one machine to call a procedure on another machine over the network, making the remote call seem like a local one. The client sends a request with parameters to the remote server, which executes the procedure and returns the results. This process involves marshalling parameters into messages and unmarshalling responses, abstracting the network details from the developer. RPCs facilitate building micro-service based distributed applications without requiring the programmer to manage complex communication technicalities. Popular RPC frameworks that implement this model include gRPC [103] and Apache Thrift [95] among others.

3.2.2 Remote Direct Memory Access

Remote Direct Memory Access (RDMA) is a technology that enables one machine to directly access the memory of another machine over the network without involving either system's operating system, CPU, or cache. This leads to high-throughput, low-latency data transfers since the transfer bypasses the kernel networking stack and avoids unnecessary memory copying. RDMA achieves this by using specialized network interface controllers that handle data operations independently. RDMA is widely used in high-performance computing (HPC), big data analytics, cloud computing, AI, and machine learning distributed training, video streaming, and database systems where low latency and high bandwidth are critical. Examples of RDMA implementations include InfiniBand, RDMA over Converged Ethernet (RoCE), and iWARP, which enable fast data transfer in environments like supercomputing clusters, cloud data centers, and storage networks, accelerating application performance and scaling capabilities.

3.2.3 Thallium Framework

Thallium is an RDMA framework that provides a modern C++14 interface to simplify building high-performance distributed applications with remote direct memory access. It abstracts the complexity of low-level RDMA networking through its core `thallium::bulk` feature, which exposes memory regions for zero-copy, low-latency data transfers directly between remote systems. By combining this with RPC capabilities, efficient serialization, and threading support via Mercury and Argobots, Thallium enables developers to seamlessly integrate RDMA into their applications using intuitive,

type-safe C++ APIs. This approach allows for high-throughput remote memory access with minimal CPU overhead and supports building scalable, low-latency communication protocols and data services. Overall, Thallium lowers the barrier to RDMA adoption by blending the expressiveness of RPC with the efficiency of RDMA, making it a powerful tool for scalable HPC and data-intensive services.

3.3 Serialization Overhead in Apache Arrow

To establish our baseline, we built a client-server application that executes SQL queries on the server using DuckDB [86] and transports Arrow record batches holding query results over plain Thallium RPCs. Apache Arrow record batches must be serialized into a contiguous buffer to be able to send them through the RPC response. This serialization process incurs copying Arrow column buffers in-memory and laying them out contiguously. We execute queries to select all the columns in the dataset and observe that about 30% of the end-to-end RPC duration is spent in serializing a record batch, and only about 0.0004% of the duration is consumed in deserialization, as it is a zero-copy operation in Apache Arrow. This work aims to remove the serialization overhead by leveraging an RDMA-based network stack for Arrow data transport.

3.4 Thallus: Design and Implementation

Using the Thallium framework, we design and implement Thallus, an RDMA-based data transport protocol for Apache Arrow. Our protocol has 2 stages: First,

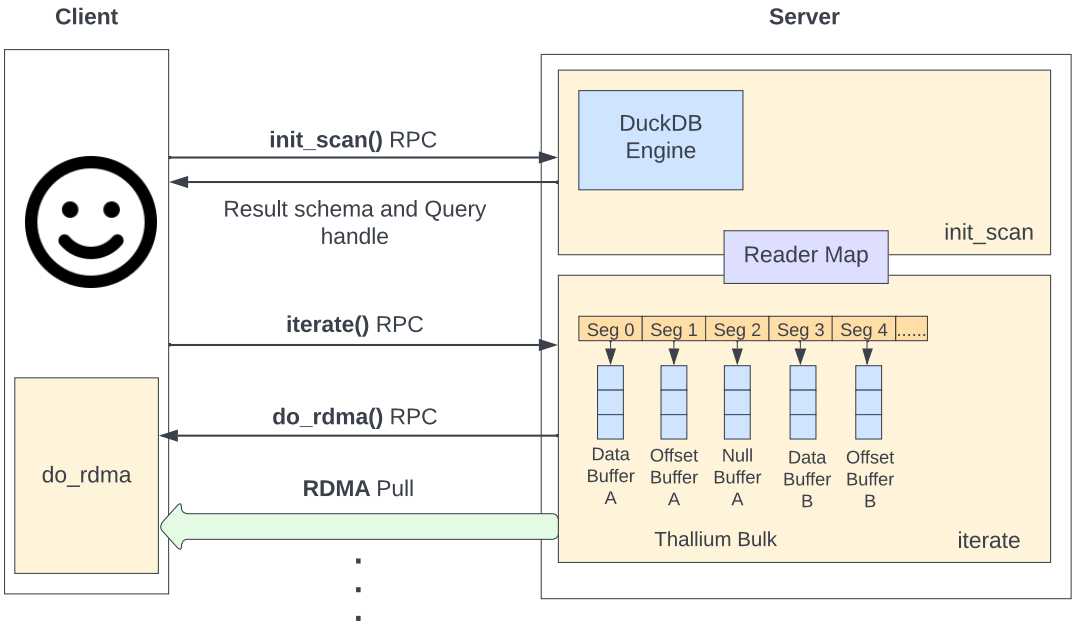


Figure 3.1: Control flow diagram showing the high-level operation of Thallus.

we initiate a query execution using a query execution or database engine; Second, we transport the query results batch by batch to the client. We use the Thallium RPC interface to exchange metadata and control information while using the Thallium RDMA interface to transport data buffers.

Upon connecting to a server, the client invokes an `init_scan` procedure on the server with a SQL query and a dataset path as arguments. Inside this procedure, a query execution session is instantiated, and a handle to a result cursor is returned to the client as a response. The client then uses this handle to invoke an `iterate` procedure on the server, which iterates over the query cursor, producing results batch by batch, and for every batch, invokes a `do_rdma` procedure on the client, that pulls the batches from the server to the client. After all the batches are pulled into the client, a `finalize`

procedure is invoked to clean up resources and stop the server. Figure 3.1 summarizes the high-level operation of Thallus in an implementation with DuckDB as the query execution backend. Next, we discuss each of the procedures mentioned above in detail.

3.4.0.1 *Server*: init_scan

This procedure takes a SQL query and a dataset path as input arguments. The procedure starts execution by instantiating a `DuckDBEngine` instance, a class containing methods for creating and querying a table in DuckDB. We can use a similar interface to leverage any other Arrow-native query execution engine, such as Polars [10] and Velox [83]. Then, a view is created over the dataset, the SQL query is parsed, and an optimized physical plan is generated for execution. Upon initiating the query execution, DuckDB returns a query cursor instance. Arrow provides a `RecordBatchReader` interface for streaming access to Arrow record batches. We extend this interface to create a `DuckDBRecordBatchReader` interface that, upon every iteration, uses the DuckDB query cursor to generate DuckDB data chunks and converts them to equivalent Arrow record batches. We achieve this conversion in a zero-copy manner using the Arrow C Data Interface. Finally, we store a pointer to this reader instance in a map, which we refer to as the reader map, using a unique UUID, read the result's schema from the reader instance, and send both back to the client as a response.

3.4.0.2 *Server*: iterate

This procedure is used to iterate over the `DuckDBRecordBatchReader` and produce result Arrow record batches. It takes the record batch reader UUID as its only input argument to identify a particular query execution instance in a multi-tenant environment. It starts execution by looking up the UUID in the reader map to find the reader instance for the client. It then iterates over the reader to produce result Arrow record batches, that are sent to the client over RDMA.

For the RDMA operations, we use the Thallium "bulk" interface, as it provides access to remote memory regions. The users must allocate a list of segments, where each segment points to a distinct memory region. These segments are then prepared for remote reads and write through an "expose" operation, which generates a bulk handle. The bulk handle is a descriptor for an RDMA-ready pinned remote memory region that can be serialized and passed around through RPC arguments and responses and read for accessing remote memory regions.

In Apache Arrow, a record batch is a collection of columns, where each column is composed of 3 buffers representing the data values, offsets, and null masks. We allocate 3x the number of segments as there are columns and for every i th column, we map its data, offset, and null buffer to the $3i$ th, $3i + 1$ th, and $3i + 2$ th segment, respectively. While mapping the buffers to the segments, we keep storing the sizes of each of the data, offset, and null buffers in 3 separate vectors. We expose these segments as a read-only Thallium bulk and invoke the client-side `do_rdma` procedure with the bulk

handle, the buffer sizes vectors, and the number of rows in the batch. The buffer size vectors must be shipped to the client for it to allocate a similar layout of buffers as on the server to pull the server-side buffers into the client-side ones in a one-to-one manner. When the reader is exhausted and no more batches can be produced, the procedure returns.

3.4.0.3 *Server*: finalize

The `finalize` procedure is invoked after the client has finished pulling all the batches from the server and needs to terminate the data transport operation. This procedure frees the allocated buffers and the reader map and finalizes the Thallium engine.

3.4.0.4 *Client*: do_rdma

This procedure pulls the Arrow record batches from the server to the client. It is invoked by the server-side `iterate` procedure with the number of rows, vectors of buffer sizes, and the remote bulk handle as arguments. It then uses this information to allocate buffers for the client to store data fetched from the server and exposes these buffers as a write-only local bulk. It then pulls the remote bulk into the local bulk using RDMA. The bulk pulling is a scatter-gather operation where the server-side buffers are fetched into the corresponding client-side buffers. Once all the buffers are brought into the client, they are associated with buffer sizes and data types (already available on the client) to instantiate column structures. These column structures are then wrapped

with the result schema to create an Arrow record batch. The record batches are then written to an output stream, such as a parquet file.

3.5 Evaluation

We benchmark Thallus by executing SQL queries that generate result sets of different sizes and use a pure Thallium RPC-based implementation as our baseline. We measure the transport and end-to-end query execution duration using Thallus and compare it with our baseline. We measured the transport duration only by eagerly collecting all the results in the server memory first and then letting the client read them.

As shown in Figure 3.2 and Figure 3.3, we observe that Thallus is up to 5.5x faster than Thallium RPC in data transport durations and achieves up to 2.5x performance gain in end-to-end query execution duration. As can be seen from both figures, the relative performance gain of Thallus over Thallium RPC diminishes with the reduction in the size of the result set. This is because when the batch size transferred reduces, the constant overheads of doing RDMA, such as buffer allocation and memory pinning, become significant.

3.6 Future Work

As future work, we would like to explore the use of Data Processing Units (DPUs) such as NVIDIA BlueField, to handle operations such as serialization, deseri-

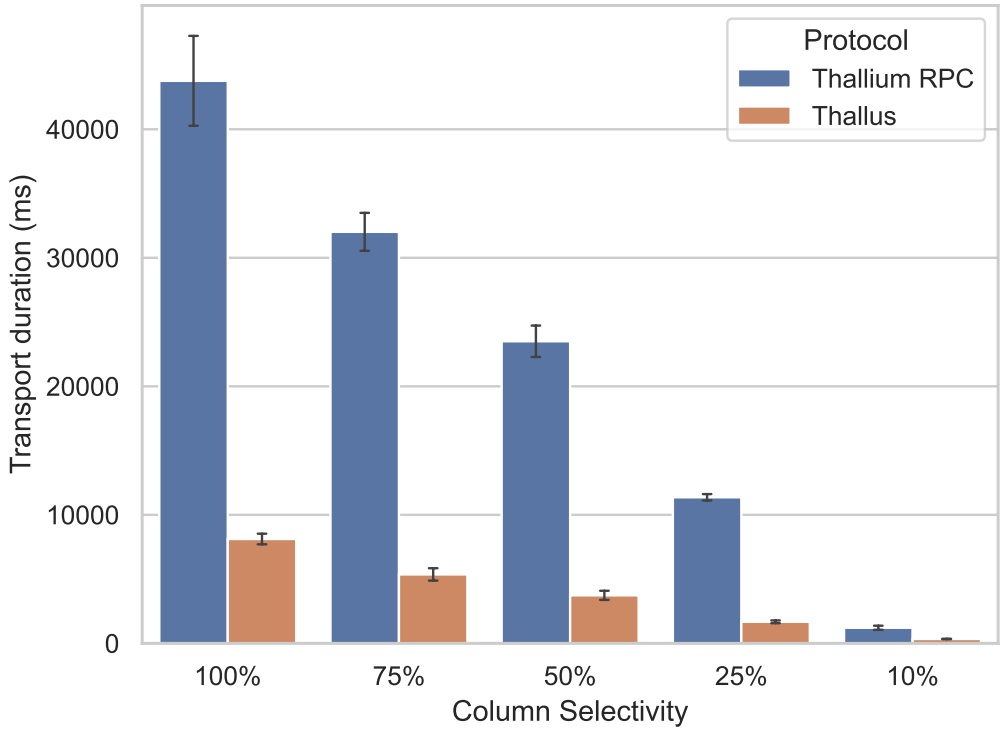


Figure 3.2: Comparison of data transport duration using Thallus versus Thallium RPC in column selectivity experiments.

alization, compression, and decompression. Offloading these tasks to DPUs can significantly reduce the processing burden on the main CPU, resulting in faster data transfers and lower latencies. In addition to these data transformation tasks, we also plan to investigate implementing the entire remote procedure call (RPC) process on the DPU. By leveraging DPUs for these infrastructure-related operations, our system could better utilize available hardware resources, dedicating more CPU cycles to compute-heavy tasks. These strategies aim to adopt a heterogeneous compute environment and improve

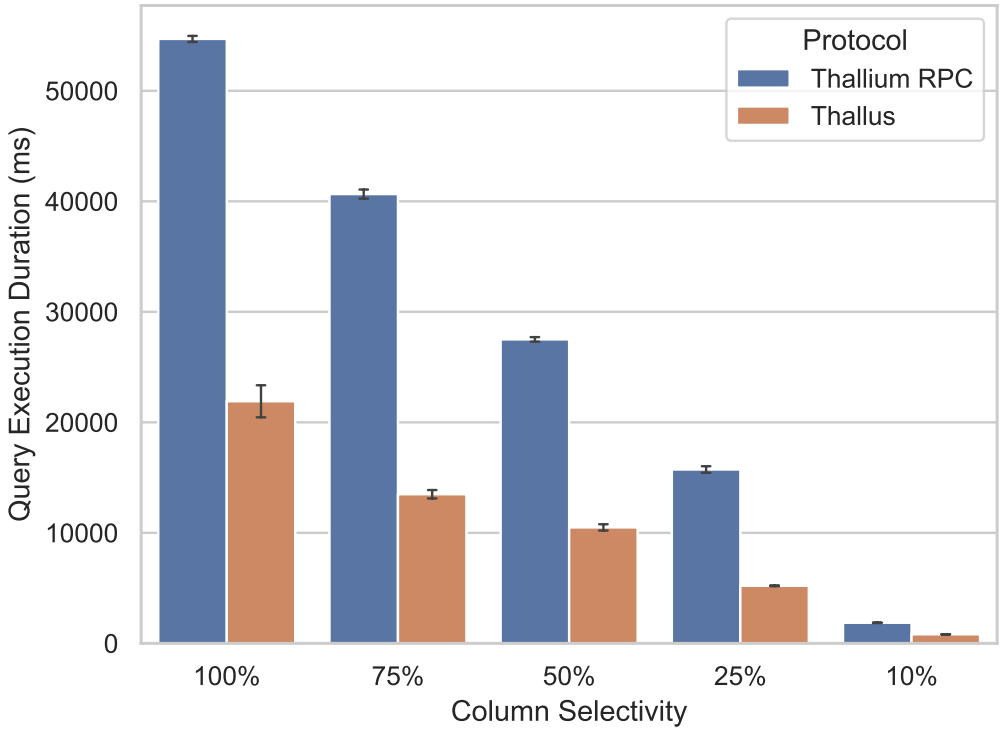


Figure 3.3: Comparison of query execution duration using Thallus versus Thallium RPC in column selectivity experiments.

overall system performance and scalability.

3.7 Conclusion

In this paper, we explore using RDMA to accelerate the transport of columnar data in distributed data processing systems. We begin by measuring the overhead caused by the expensive serialization of Apache Arrow data when transported over the wire using a traditional TCP/IP-based RPC framework. To tackle the serialization

overhead, we design and implement Thallus, an RDMA-based columnar data transport protocol using the Thallium framework from the Mochi ecosystem to transport Apache Arrow data over Infiniband. Internally, Thallus uses RPCs for control operations and RDMA for data operations. We structure our implementation as a client-server model where queries are executed in a DuckDB instance running on the server, and results are sent back to the client using Thallus. We benchmark Thallus against a pure RPC-based implementation, which was also written using the Thallium framework. Thallus exhibits up to 5.5x improvement in data transport performance and up to 2.5x improvement in end-to-end query execution performance over TCP/IP-based Thallium RPCs on OLAP-style columnar workloads. Our studies show that RDMA can be a viable and more performant alternative to TCP/IP for data transport in modern data processing applications.

Chapter 4

Characterizing the Performance of Approximate Nearest Neighbor Searches

4.1 Introduction

Vector search implemented through Approximate Nearest Neighbor (ANN) algorithms, is foundational in AI applications for efficiently retrieving semantically similar items from large high-dimensional data collections. Unlike traditional search which relies on exact keyword matches, vector search transforms multi-modal data such as text, images, or user interactions into dense numeric embeddings, capturing their semantic meanings. This enables systems to find relevant results based on context rather than simple overlap of words or metadata. The importance of vector search is especially evident in AI pipelines requiring real-time, contextual retrieval over massive datasets. For example, Retrieval Augmented Generation (RAG) systems use vector search to fetch

semantically relevant documents from a knowledge base, supplementing large language models so they can generate accurate, grounded responses. In practice, a RAG-based chat engine might use vector search to retrieve enterprise documents that clarify an employee's technical question, thereby improving the chat engines accuracy and reducing hallucination. In recommendation engines, such as Deep Learning Recommendation Models (DLRM), user and item features are embedded into vectors. Vector search enables these systems to rapidly and accurately suggest products, videos, or friends that align with user preferences by calculating similarities in the embedding space. This powers platforms from e-commerce sites to music and movie streaming services, where recommendations are based on latent user signals rather than explicit tags. Other applications include image and video recognition, where feature vectors enable instant retrieval of visually similar items from millions of assets, useful in content management and social media. Also in multi-modal and hybrid search, where queries may combine text and image input, unified by their vector representations. These use cases demonstrate how ANN-based vector search not only enables advanced semantic search and content discovery but also makes scalable, real-time AI services viable on enormous datasets underscoring its central role in modern artificial intelligence.

As AI applications like RAG, DLRM [76], and large-scale recommendation or retrieval engines scale to billions of vectors, memory and computational demands rise sharply making traditional CPU-based approaches a bottleneck. Accelerators with powerful compute units and high bandwidth access to fast memory is critical in improving the performance of processing hundreds of thousands of vector search queries per sec-

ond. To that end, understanding the performance characteristics of these vector search workloads is highly important, as that understanding would guide research in building the next generation of vector search algorithms, software stacks, custom hardware accelerators, compression techniques, and overall vector database system architectures for vector retrieval to stay feasible and efficient as data scales and application demands increase. In this work we perform a detailed analysis of the performance characteristics of vector search workloads in terms of compute hotspot, top down analysis, throughput, scalability, compute and memory utilization, and capacity requirements.

4.2 Background

4.2.1 LLMs and Retrieval Augmented Generation (RAG)

Large Language Models (LLMs) are typically trained on massive corpora of text data scraped from the internet, spanning a wide range of general knowledge. However, while these models demonstrate broad level of competence, they often lack deep expertise in specific domains, essentially becoming "jack of all trades, master of none." In many real-world applications, such as healthcare, law, or finance, domain-specific knowledge is essential to generate accurate and reliable responses.

There are two primary strategies for enhancing LLMs with specialized knowledge: fine-tuning [48] and Retrieval-Augmented Generation (RAG) [69]. Fine-tuning involves retraining a pre-trained LLM on domain-specific datasets. While this approach can improve accuracy, it is resource-intensive, requiring significant computational in-

frastructure (e.g., GPUs) and comes with trade-offs such as longer training times, more power consumption, reduced model generality, and the risk of locking the model into a static knowledge snapshot.

RAG offers a more flexible and cost-effective alternative. It enables the use of a pre-trained LLM without modifying its weights. In a RAG system, domain-specific documents are first tokenized, embedded into vector representations, and stored in a vector database. When a user submits a query, semantically relevant embeddings are retrieved from the database and passed as contextual input alongside the query to the LLM. The model then generates a response that incorporates both the retrieved context and the query itself. Figure 4.1 shows the architecture of a typical RAG system.

This approach has several advantages: it eliminates the need for expensive retraining, supports dynamic and up-to-date knowledge integration, and preserves the general capabilities of the base LLM. As a result, RAG has emerged as a practical and scalable solution for deploying LLMs in specialized applications without sacrificing flexibility or performance.

4.2.2 Vector Database and Approximate Nearest Neighbor (ANN) Search

Vector databases are essential infrastructure for modern machine learning systems, particularly those leveraging embedding-based retrieval methods such as Retrieval-Augmented Generation (RAG). In these approaches, unstructured data including text, images, or source code is first transformed into high-dimensional vector representations,

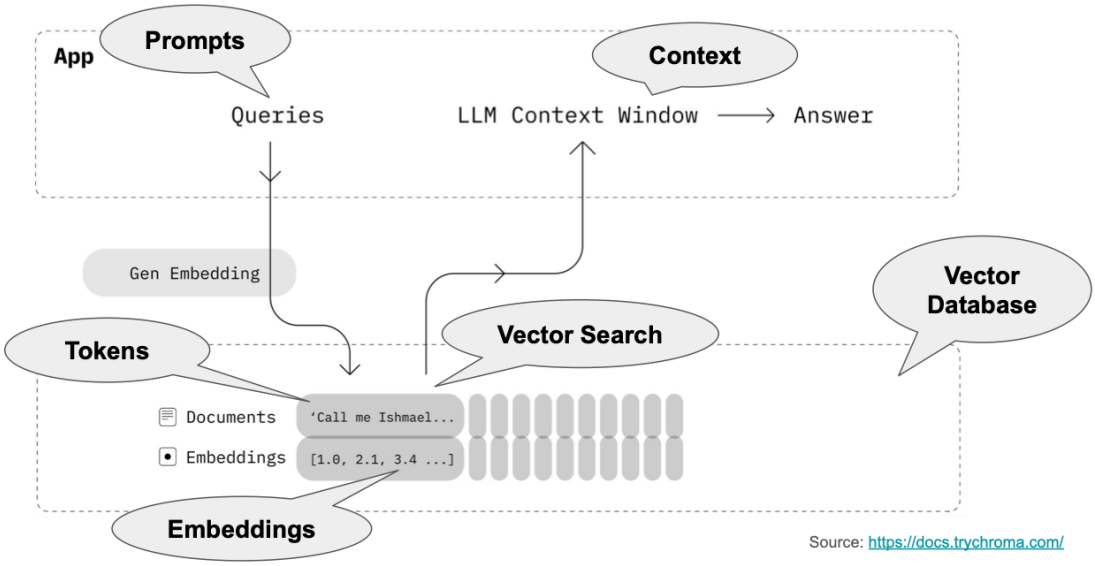


Figure 4.1: Architecture of a Retrieval Augmented Generation (RAG) system

known as embeddings. These embeddings encode semantic relationships among data items, allowing similarity assessments based on geometric proximity (such as cosine similarity or Euclidean distance) within a vector space.

A vector database stores these embeddings along with associated metadata, enabling efficient storage, indexing, and retrieval of data points based on their semantic similarity rather than exact matching. Embeddings are typically generated by passing tokens (such as words or sentences) through a trained embedding model, which outputs numerical vectors situated in an N-dimensional space. Each token is thus represented as a point in this space, with semantically similar items located closer together. Examples of some popular embedding models include `text-embedding-large-3` from OpenAI and `embed-english-v3.0` from Cohere.

When a query is presented, its embedding is computed and compared with

those stored in the database using similarity search algorithms. Due to the potentially vast number of tokens ranging into the billions, exhaustive (exact) nearest neighbor search becomes computationally prohibitive. To address this, vector databases employ Approximate Nearest Neighbor (ANN) algorithms that trade off some accuracy for significant improvements in search speed and scalability. Major categories of ANN algorithms include graph-based methods (such as HNSW, Hierarchical Navigable Small World [71]; NSG, Navigating Spread-Out Graph [54]), cluster-based methods (such as IVF, Inverted File Index [30]), and hashing-based methods (such as LSH, Locality Sensitive Hashing [59]). Additionally, for extremely large datasets that exceed system memory, specialized algorithms like DiskANN [64] and Vamana are used to maintain performance at scale. Vector databases provide a robust and scalable foundation for high-dimensional similarity search, powering a wide range of AI applications including semantic search, recommendation systems, and retrieval-augmented generation tasks. Examples of widely-used vector databases include Milvus [102], Qdrant [13], pgvector [8], and Pinecone [9].

4.2.3 Hierarchical Navigable Small World (HNSW) Algorithm

The Hierarchical Navigable Small World (HNSW) algorithm is a state-of-the-art algorithm for approximate nearest neighbor (ANN) search designed for execution on CPUs. HNSW builds on the concept of navigable small world graphs by organizing data into a multi-layered graph structure. In this graph structure, the top layers are sparse and allow for long-range connections across the dataset, while the bottom layers

are denser and capture local neighborhood connections. This layering enables fast and scalable searches, as queries can quickly traverse through sparse levels and then refine their search in denser layers, making HNSW remarkably efficient for large databases and vector search applications. The structure of the HNSW graph index is show in figure 4.2.

The index building process in HNSW involves inserting embeddings one at a time, assigning each a randomly determined maximum layer, and connecting them to existing points at each layer based on proximity. The algorithm uses several parameters to control this process: `M` (or max_neighbor_count) determines the maximum number of edges (connections) each node can have in each layer, influencing both the density of the graph and its search speed and accuracy; `ef_construction` controls how many candidate neighbors are considered during the insertion of a node; a higher `ef_construction` can greatly improve index quality and recall but increases the computational overhead and build time. This process uses a greedy search to identify the best neighbors and applies advanced heuristics to build shortcuts between clusters, ensuring robust navigability and efficient future queries.

During the search process, the HNSW algorithm starts at the topmost layer of the hierarchy and follows a greedy strategy to find the node nearest to the query. At each layer, it explores the neighboring nodes, seeking the one closest to the query until it reaches the base layer, where most of the data resides. The key search parameter is `ef_search`, which sets the size of the candidate list during search. Increasing `ef_search` results in more exhaustive searches and higher accuracy (recall), albeit at reduced query

speed. In summary, higher `M` and `ef_construction` values lead to denser, more accurate, and resource-intensive indices; increasing `ef_search` yields better recall but at the cost of longer search times, allowing users to balance accuracy, speed, and resource usage as needed for their specific application.

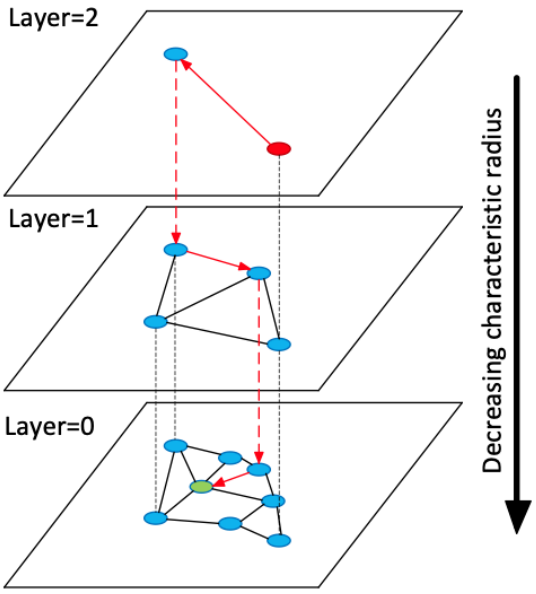


Figure 4.2: Structure of the Hierarchical Navigable Small World (HNSW) graph

4.3 Performance Characteristics

We explore the performance characteristics of the Hierarchical Navigable Small World (HNSW) and Inverted File (IVF) approximate nearest neighbor (ANN) algorithms across both CPU and GPU platforms. We analyze memory requirements and indexing efficiency, with a focus on how these metrics scale as dataset sizes increase and key algorithm parameters such as the connectivity parameter "M" and the con-

struction parameter "ef_construction" in HNSW are varied. The investigation extends to multi-core processor environments, assessing the effects of parallelization strategies and batch sizes on throughput and latency. At the hardware level, we examine critical factors including data movement between DRAM and compute units, operational intensity, and utilization of SIMD units. Additionally, we evaluate how closely each algorithm approaches the theoretical peak memory bandwidth of the underlying platform. By providing a detailed analysis of the interplay between ANN algorithmic design and system architecture, this work offers practical guidelines for optimizing high-performance vector search workloads that underpin modern retrieval-augmented generation and other AI applications.

4.3.1 Experiment Setup

For our CPU-based studies using the HNSW algorithm, we conduct experiments on a Supermicro server configured in a dual-socket setup, with each socket equipped with a 4th Generation Intel Xeon Scalable Processor (Sapphire Rapids). Each processor came with 24 physical cores, with hyperthreading enabled, resulting in a total of 48 cores across the system. The CPU SKU was Intel Xeon Gold 5418Y having a clock frequency of 2.0 GHz. The memory subsystem consisted 8 DDR channels per socket, populated in a 1 DPC (1 DIMM / channel) configuration using 32 GB DDR4 DIMMs from Micron having a frequency of 4800 MT/s leading to a total DRAM capacity of 256 GB and theoretical bandwidth of about 300 GB/s. Our machine ran Ubuntu 24.04.1 LTS on top of Linux kernel version 6.8.0. For our experiments, we used only used the

physical cores of a single socket and also pinned the memory to be used from that particular socket using `numactl`.

Among the many vector databases and ANN search libraries available out there, we chose the FAISS [51] library from Facebook for our experiments. We use version 1.11.0 of FAISS, the latest release of FAISS at the time of writing this document. The reasons for using FAISS are as follows:

- FAISS provides implementations of popular ANN search algorithms with platform-specific optimizations and is the state-of-the-art in ANN search libraries.
- FAISS is not a full-fledged vector database and only an embeddable ANN search library, hence there is no database control plane overhead involved. This makes FAISS ideal for benchmarking and profiling when studying performance characteristics.
- FAISS is open-source and maintained by Meta along with the broader ANN search community. They make releases regularly and keep adding new optimizations and features.

We use `g++` version 13.3.0 for compiling FAISS while enabling AVX-512F and FMA (Fused-Multiply-Add) support. For the highest BLAS performance, we use Intel MKL (Math kernel library) instead of OpenBLAS. Additionally, we use the "-Ofast" flag for the highest level of optimizations. We also use the same compiler configurations to compile our benchmark wrapper code written using FAISS. We used the Yandex Text-to-Image vector embeddings dataset [16] from Yandex Research for our performance

characterization experiments. This dataset contains 1 billion embeddings, each with a dimension of 200 and 100K query vectors. The recommended distance metric for this dataset was Inner Product (IP) and hence we used the same. For our experiments, we used slices of the dataset ranging from 1M to 128M increasing in powers of 2. We always sliced the dataset from the beginning. All the ANN searches performed for the experiments were carefully configured to yield an recall (accuracy) of approximately 95% at a top K of 10 using the HNSW search parameter `ef_search`.

4.3.2 HNSW Index Characteristics

We study characteristics of HNSW indexes built using FAISS such as index file size, index build duration, and variation of index file size and build duration with the number of neighbors per node of the graph which is controlled by the parameter M. From table 4.1, we see that the file size of the indexes scale linearly with the number of vectors used to create the index or the dataset size (N) while the index build duration, scale a little super-linearly with the dataset size. Additionally, we perform the same study by varying the number of neighbors per graph node (M). From table 4.2, we observe that both file size and build duration increase sub-linearly with respect to the parameter M.

4.3.3 CPU Hotspot Profiling

We run `perf record` on a FAISS HNSW search on 64M dataset to find out how the different stages of the search compose the total search duration. The

Dataset Size (N)	Index File Size	Build Duration
1M	626M	6.43s
2M	1.3G	14.60s
4M	2.5G	33.98s
8M	4.9G	77.33s
16M	9.8G	178.12s
32M	20G	434.62s
64M	40G	1078.58s
128M	79G	2009.10s

Table 4.1: File size and build duration of HNSW index with M=32 on varying slices of the Yandex Text-to-Image dataset

Dataset Size	M	Index File Size	Build Duration
1M	16	504M	3.80s
1M	32	626M	6.10s
1M	64	870M	7.67s
1M	128	1.4G	9.45s
1M	256	2.3G	11.62s
1M	512	4.2G	14.81s
1M	1024	8.1G	20.22s

Table 4.2: File size and build duration of HNSW index with varying M on a 1M slice of the Yandex Text-to-Image dataset

main components of a HNSW search include: distance calculation, graph traversal, and sorting (to find the closest M neighbors in every layer). As seen from the `perf report` screenshot in figure 4.3, about 42% the time is spent in L2 distance calculations (`faiss::fvec_L2_sqr_batch_4`; `faiss::fvec_L2_sqr`), navigating the HNSW graph index takes about 15.5% (`faiss::search_from_candidates`; `faiss::neighbor_range`), and sorting comprises about 6%. Rest of the time is spent in reading out the HNSW index into the memory from disk which involves disk i/o and deserialization (memory copies). These statistics indicate that distance calculation is

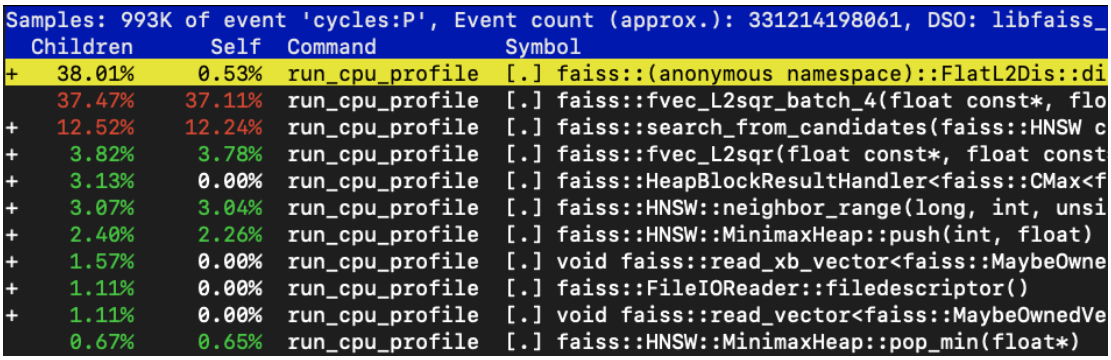


Figure 4.3: Breakdown of the FAISS HNSW search function based on statistical profiling

the hotspot of the HNSW search (just considering the in-memory part of the search) and is an important candidate for acceleration.

4.3.4 Top Down Micro-architecture Analysis

Table 4.3: Top-Down Micro-architecture Analysis (TMA) of HNSW search

Metric	Value (%)
tma_backend_bound	57.0
tma_bad_speculation	2.7
tma_frontend_bound	15.3
tma_retiring	25.0
tma_core_bound	13.0
tma_heavy_operations	6.3
tma_light_operations	18.8
tma_memory_bound	44.0
tma_branch_mispredicts	1.9
tma_fetch_latency	7.9
tma_machine_clears	0.8
tma_fetch_bandwidth	7.3

We perform a Top down analysis of the same HNSW search run as done in figure 4.3.3 to understand the nature of the workload. Top down analysis categorizes a workload in terms of its backend boundness, frontend boundness, bad speculation, and

retiring percentages. We organize the top down numbers as reported by `perf stat` in table 4.3. As can be seen, the HNSW search workload is mostly backend bound (57%), and within backend boundness, it is mostly memory bound (44%). This means that 44% of the pipeline slots are stalled due the workload on waiting on memory operations and 13% of the pipeline slots are stalled due to the unavailability of compute (divider, floating point unit, SIMD engine) resources. Being memory bound indicates that improving the memory subsystem with higher memory bandwidth (more memory channels), faster DDR memory, larger caches and good prefetchers would result in a significant improvement in the throughput of the system. We also see that this workload does not suffer from too much branch mispredictions and machine clears due to bad speculations. This indicates the availability of good branch predictors and sophisticated control logic on the Intel Sapphire Rapids CPU. Finally, the frontend boundness of this workload is about 15%, which is totally reasonable and does not show any issue with the fetch and decode stages of the CPU pipeline.

4.3.5 QPS / Throughput Scaling

We study how the throughput (queries / second) of ANN searches scale with the dataset size and query batch size. From figure 4.4, we observe that QPS drops exponentially with the increase in dataset size from 1M to 128M. We also observe that when the same experiment is repeated by increasing query batch size from 100 to 100000, higher QPS is observed for every dataset size. Figure 4.5 shows the exact exponential at which the performance drops with increasing dataset sizes. Such a power law equation

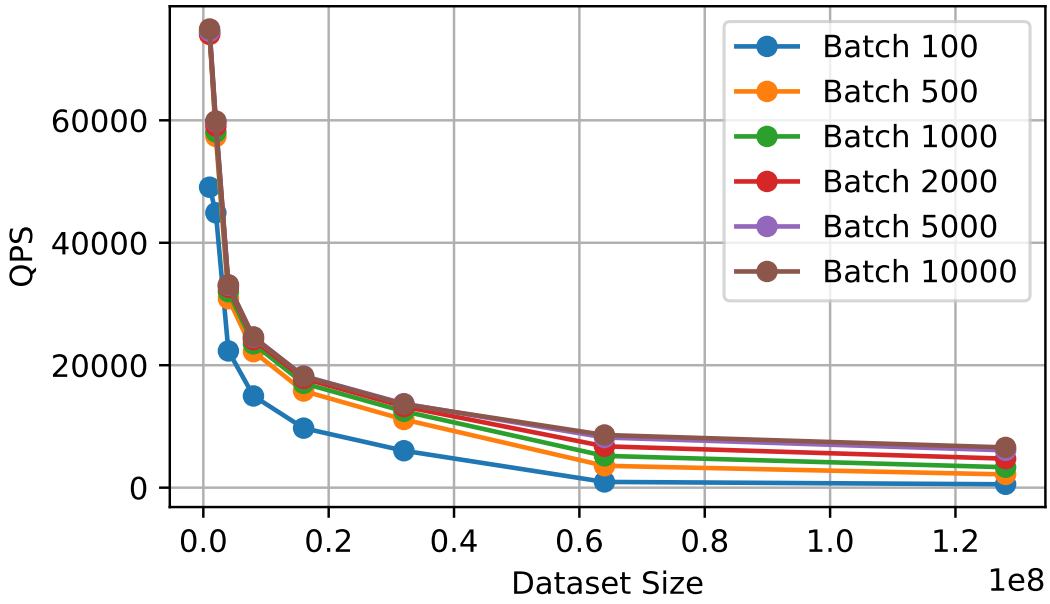


Figure 4.4: QPS scaling with varying dataset size and query batch size

can be useful for estimating performance for any required dataset and query batch size.

4.3.6 Memory Bandwidth Scaling

We study how the systems memory bandwidth get utilized by these workloads and its scaling behavior with varying number of cores, dataset sizes, and query batch sizes. We begin by baselining the systems memory bandwidth using the Triad workload in the STREAM benchmark suite. The STREAM Triad operation is a simple but important compute kernel used in memory bandwidth benchmarking. It performs the computation:

$$c[i] = a[i] \times k + b[i]$$

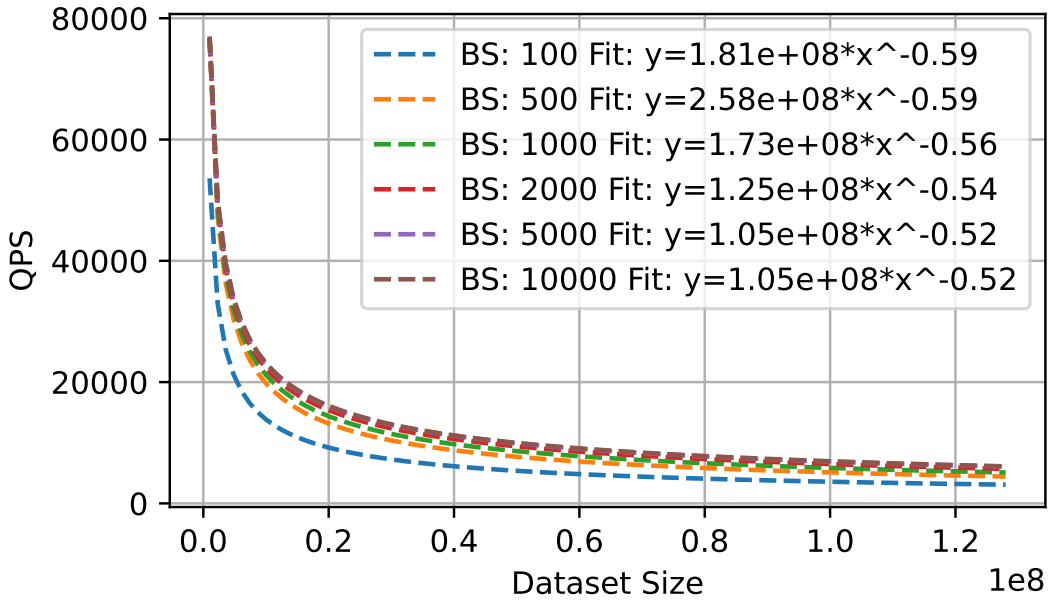


Figure 4.5: QPS scaling power law with dataset size and query batch size

in a loop across three arrays `a`, `b`, and `c` of a given size, where `k` is a scalar constant. The kernel involves three memory loads and one memory store per iteration. Two of the memory loads are required to read the values of `a[i]` and `b[i]` from memory, and the third load is required to read the value of `c[i]` into the cache for a write-back based store operation. On the compute side, one floating-point multiply and add operation occurs per iteration. Figure 4.6 shows the peak memory bandwidth of our test system. The total bandwidth (read + write bandwidth) of our system peaks out at about 134 GB/s when using all the physical cores.

We run our ANN search workload and measure the bandwidth using the Intel PCM (Performance counter monitor) tool. As shown in figure 4.7, using all the physical cores of a single socket, our workload fetched a bandwidth of about 100 GB/s, which is

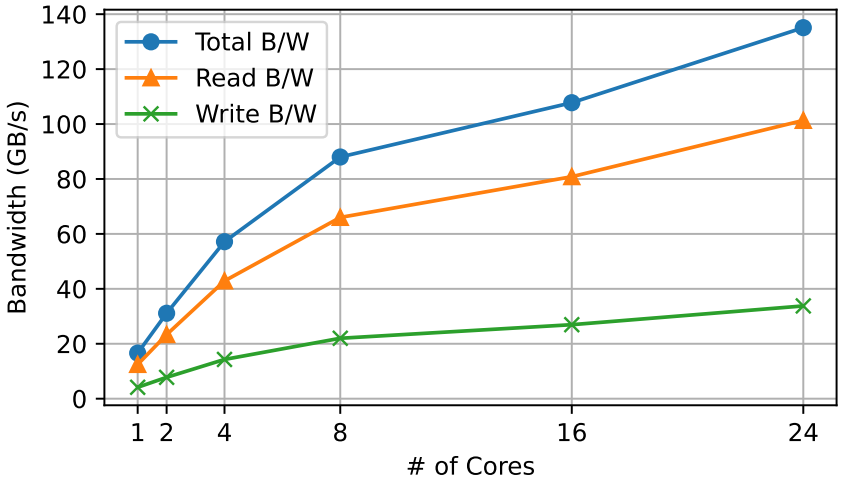


Figure 4.6: STREAM triad bandwidth scaling of our test system with the number of cores

about a 75% utilization of our peak bandwidth. We also see that with increasing number of cores, the memory bandwidth utilization increases, which proves that the ANN search workload is memory bandwidth bound. Additionally, we study how the memory bandwidth scales when varying the dataset size and query batch size. One would expect the memory bandwidth utilization to be constant for any reasonable dataset size and that is what we find as well as shown in figure 4.8. On the other hand, we observe that with increase in the query batch size, the bandwidth utilization increases until a batch size of about 5000 and then peaks out at 100 GB/s. This is shown in figure 4.9. This observation is inline with the fact that FAISS uses a thread-pool to dispatch a batch of queries in parallel to the HNSW index and a certain amount of thread-pool depth or queue-depth is necessary to sustain the peak memory bandwidth.

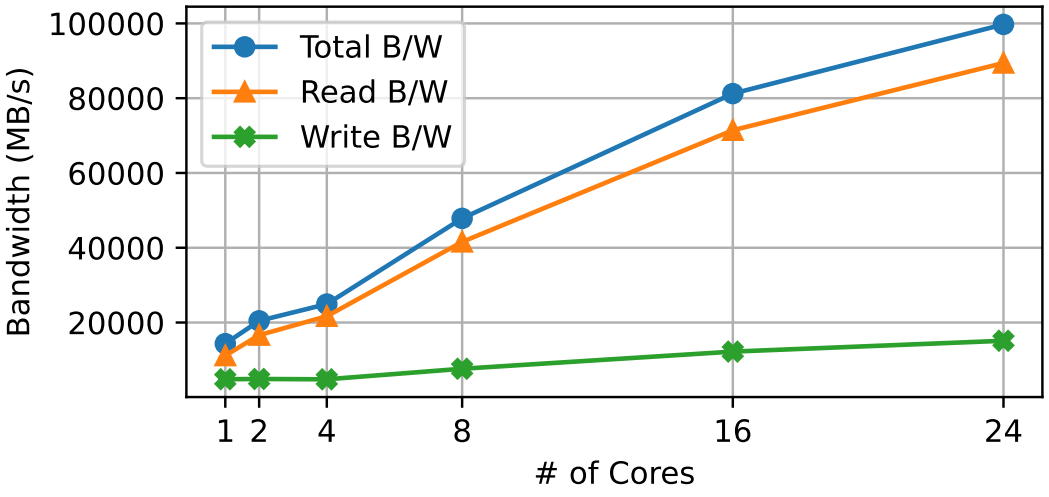


Figure 4.7: Memory bandwidth scaling with varying number of cores in FAISS HNSW search

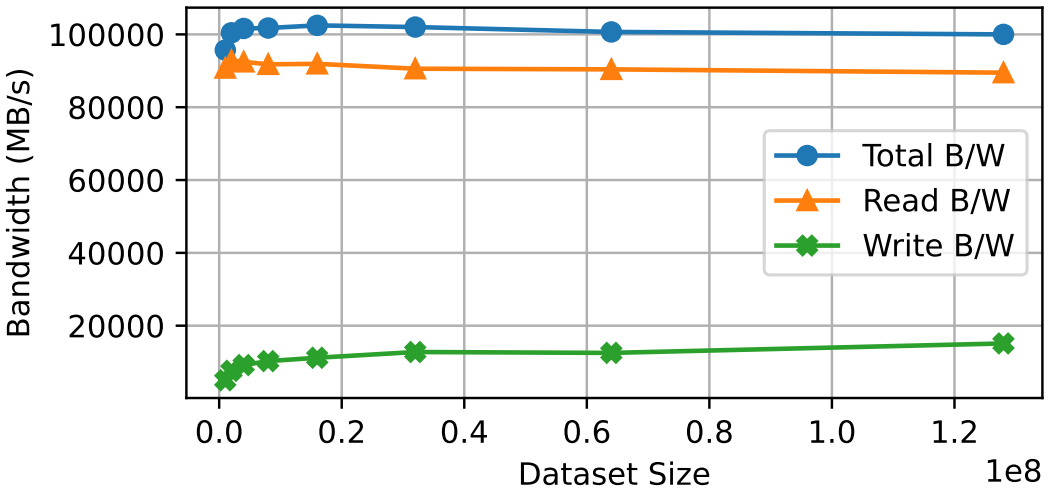


Figure 4.8: Memory bandwidth scaling with varying dataset sizes in FAISS HNSW search

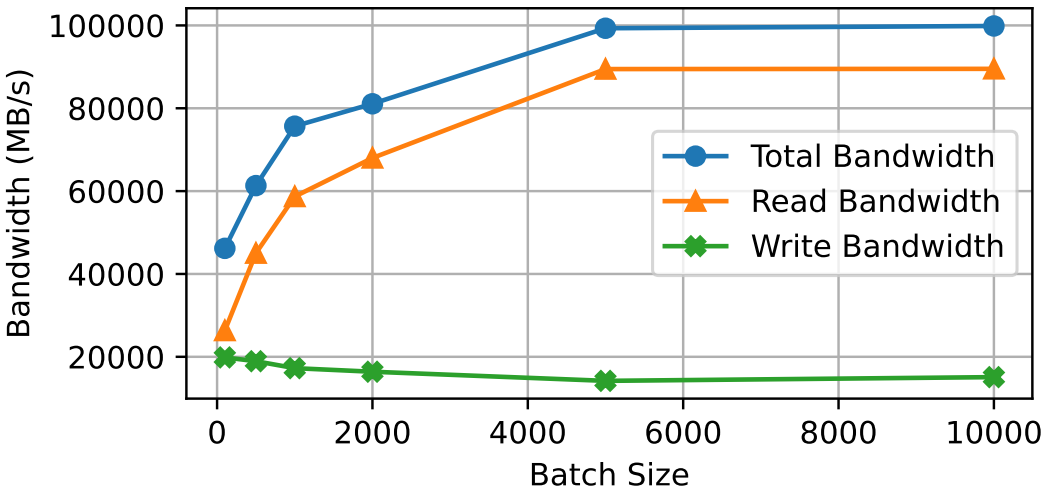


Figure 4.9: Memory bandwidth scaling with varying query batch sizes in FAISS HNSW search

4.4 Conclusion

Approximate nearest neighbors (ANN) algorithms form the core of many AI applications ranging from RAG systems to recommender systems. Being a crucial component of AI applications, the execution of these algorithms need to be highly performant for it to not become a major bottleneck. Several industry and academic researchers are working on to accelerate these algorithms using modern hardware such as sophisticated CPUs, GPUs, and custom XPUs. Understanding the performance characteristics of a workload is highly important for developing techniques to accelerate its performance and efficiency. In this work we characterize the performance of a common ANN search algorithm, Hierarchical Navigable Small World (HNSW) using a highly-efficient implementation of it in the Facebook FAISS library. We characterize the throughput

and memory bandwidth utilization of this workload in terms of its scaling behavior with respect to number of cores, dataset size, and query batch size. We also perform a CPU hotspot and Top down micro-architecture analysis of this workload to understand the nature of this algorithm. Our experiments confirm the memory boundedness of ANN search algorithms and suggest CPUs with more memory channels and faster DDR memory is desirable to run these algorithms at high throughput. The CPU hotspot analysis reveals that distance calculations dominate the duration of this workload and is a good candidate for acceleration using technologies such as AVX-512F SIMD engine or Intel AMX units or NPUs, among others. We also study the memory capacity requirements these workloads have, and find that with increase in dataset size and connectivity of the graph index HNSW, memory capacity utilization increases. This suggests that for billion scale dataset sizes, the memory capacity of present day state-of-the-art machines would fall short and require the need for memory expansion using technologies such as CXL (Compute Express Link). Overall, we hope that this study helps researchers working on accelerating ANN search performance, make more informed design choices.

4.5 Future Work

As future work, we aim to understand the performance characteristics of these workloads on GPUs and other AI accelerators such as on Intel AMX (Advanced Matrix Extensions) and ARM Ethos NPUs. Related research has shown ANN searches to perform slightly better on GPUs (using data-parallel cluster-based algorithms such as

IVF) than CPUs (graph-based; HNSW), but we argue that huge improvements in TCO can be realized by taking a hybrid approach where we execute ANN searches on Matrix Multiplication accelerators or NPUs. NPUs have access to the large DRAM capacity of CPUs while having orders of magnitude higher floating-point performance. Additionally, we plan to explore a scale-up approach of supporting billion-scale ANN searches by using CXL-attached memory expanders instead of a scale-out approach, where more machines are added but compute resources on them are utilized inefficiently. Having an in-depth understanding of the performance characteristics of ANN search algorithms on these different types of hardware platforms would guide the design decisions to design the next generation of these algorithms catered towards truly heterogeneous architectures.

Chapter 5

Future Project Ideas

5.1 How to support ANN searches on larger-than-DRAM datasets without compromising on performance ?

So far we have just looked at ANN searches on datasets that fit into the DRAM of the machine. But within hyperscalers, dataset sizes are well-beyond the DRAM capacity that today systems are capable of having. Currently, billion-scale searches are handled by scaling out; sharding the dataset across a cluster of machines. Although the total DRAM capacity of the cluster allows searching large scale datasets, the ANN search workload being inherently of low computational intensity, does not utilize the large number of CPU cores that are available in the cluster. This leads to stranded compute resources in terms of unused CPU cores.

In the past few years, technologies like Compute Express Link (CXL) [47] has emerged that allows scaling up the DRAM capacity of hosts without adding more CPU

cores. The CXL protocol allows the host CPU to access CXL-attached DDR memory over the PCIe interconnect using standard load and store instructions. Devices known as "CXL memory expanders" [82] can be attached to the PCIe slots of hosts and more memory is instantly available for use to the system. Companies like Samsung and Marvell already has CXL-attached memory expanders ready for production. These systems allow breaking the limitation of fixed DRAM capacity due to pin-count constraints and enables adding and removing memory in a flexible manner, making scaling up and down quick and flexible as a result.

We aim to study how the performance of billion-scale ANN searches would look if ran in scaled up server (using CXL-attached DRAM) versus running in a traditional scaled-out scenario. We suspect subpar performance in such a system initially as compared to a scaled-out cluster due to the higher latency of accessing CXL-attached memory and the limited bandwidth due to the PCIe bandwidth limitation. In a typical CXL-attached memory system, latency of accessing CXL memory is about 200ns, very close to the cross-NUMA memory access latency of the host. Bandwidth is limited by the bandwidth of the PCIe link; which in a PCIe Gen5 link would be 128 GBps. But being able to scale up a host instead of scaling out, should bring about a good amount of TCO benefit and greatly reduce wastage of compute resources. We believe these benefits are strong enough for us to rethink the design of these ANN search algorithms, the assumptions behind them, and modify them for high performance on systems with CXL-attached memory. Careful co-design and optimization of these algorithms for working efficiently with memory expanders would close the performance gap due to the higher

latency and limited bandwidth of CXL-attached memory.

5.2 How to use heterogeneous compute hardware to accelerate an end-to-end AI application pipelines ?

With the exponential growth of AI workloads in the past few years, modern data centers have started transitioning to having a more heterogeneous architecture. Traditionally, data centers consisted of racks of servers with CPUs, DRAM, and storage, and optionally GPUs, all connected by network switches. However, with the growing demand of accelerated training and inference of large language models, custom hardware such as NPUs (Neural processing units), GPUs, custom logic FPGAs, CPU-GPU co-located systems such as NVIDIA Grace Hopper [90], CXL-based memory expanders and accelerators, Smart SSDs, and DPUs have started to become common in data centers, especially in hyperscalers. We predict that this trend of data centers becoming more and more heterogeneous is still in very early stages and going to increase exponentially. The reasoning behind this is that heterogeneous architectures allow running parts of applications on the hardware resources that are the most efficient for that parts execution. In computer science, we have known for ever that "one size does not fit all", and heterogeneous architectures allow realizing this understanding.

Most modern CPUs today from Intel and AMD, such as their Sapphire Rapids and Turin generation of CPUs respectively, come with on-chip AI accelerators in the form of Matrix Multiply Accumulate (MAC) units; e.g. Intel AMX (Advanced Ma-

trix Extensions) [66] and Ethos from ARM. These hardware blocks are designed to perform GEMM (General Matrix Multiplication) and GEMV (General Matrix Vector Multiplication) operations on matrices using a single instruction with high floating performance. These hardware units often share the memory subsystem with the rest of the CPU starting from the last level cache. Being on-chip, with access to the memory subsystem, and accessible via the ISA, makes using these hardware blocks straight forward for the application developer. Besides on-chip accelerators, several PCIe connected inference accelerator cards have become popular such as Tenstorrent Blackhole [50], SambaNova [85], and Groq LPU. The main component of these AI inference accelerators are powerful MAC units with HBM (High Bandwidth Memory) and fast scratchpad memory in the form of SRAM. All these systems have extensive software stack support in terms of compilers, SDKs, extensions to pytorch and other ML libraries. Also, DPUs such as NVIDIA BlueField cards has become common in modern data centers and they execute network stack operations such as serialization/deserialization, compression/decompression, and RPC stack.

We aim to dive deep into these heterogeneous computing environments and rethink how AI applications such as ANN searches and Deep Learning Recommender (DLRM) systems can leverage the wide variety of compute and memory capabilities that these systems provide to realize performance and TCO benefits. For example, typical RAG systems include components such as a ANN index traversal, distance compute operations, re-ranking using LLMs, data movement in and out of vector data stores for refreshing and updating indexes. One can imagine running such a RAG system

on a heterogeneous system: CPUs performing graph traversal (graph traversal being a branchy operation with high control overhead is more suited for CPUs), on-chip or PCIe connected inference accelerators performing distance calculations in the form of dot product computations on batches of large vectors organized as matrices, GPUs running the re-ranking models, and DPUs accelerating data movement in and out of the storage devices or the entire system. Such an architecture ensures that all the different components of a complex AI pipeline runs at their full throughput as these different components get to execute on the hardware most suited for their type of workload. Additionally, different hardware units running distinct parts of an end-to-end AI application means high opportunities for pipeline parallelism and increases overall system throughput.

Chapter 6

Proposed Timeline

Following, we provide a brief timeline of expected project completion and target conferences for publication.

Project	**Completion Timeline**	**Target Conference**
Performance Characterization of Vector Search Workloads on Modern Hardware	Fall 2025	ATC 2026
Using CXL Memory Expanders for Billion-Scale ANN Searches	Spring 2026	ASPLOS 2027
Short paper on using heterogeneous architectures for accelerating AI application pipeline and Thesis writing	Summer 2026	HotInfra / HotOS 2027
Defense	Fall 2026	

Table 6.1: Timeline for completion

Bibliography

[1] Apache parquet. https://parquet.apache.org/.

[2] Arrow dataset api. https://arrow.apache.org/docs/python/dataset.html.

[3] Arrow file format. https://arrow.apache.org/docs/cpp/api/dataset.html#file-formats.

[4] Ceph object class sdk. https://docs.ceph.com/en/latest/rados/api/objclass-sdk/#sdk-for-ceph-object-classes.

[5] Hdfs block size. http://hadoop.apache.org/docs/r3.0.0/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html.

[6] Mysql. https://www.mysql.com/.

[7] Nyc yellow taxi trip record data. https://www1.nyc.gov/site/tlc/about/tlc-trip-record-data.page.

[8] pgvector. https://github.com/pgvector/pgvector.

[9] Pinecone. https://www.pinecone.io.

[10] polars. https://pola.rs.

[11] Postgresql. https://www.postgresql.org.

[12] Prometheus. https://prometheus.io/.

[13] Qdrant. https://www.qdrant.tech.

[14] Skyhook: Bringing computation to storage with apache arrow. https://arrow.apache.org/blog/2022/01/31/skyhook-bringing-computation-to-storage-with-apache-arrow/.

[15] Skyhook in apache arrow. https://git.io/JDuxJ.

[16] Yandex text-to-image. https://research.yandex.com/datasets/text-to-image-dataset-for-billion-scale-similarity-search.

[17] Google cloud dataflow, 2015.

[18] Apache orc: High-performance columnar storage for hadoop, 2018.

[19] Gandiva: A llvm-based analytical expression compiler for apache arrow. https://arrow.apache.org/blog/2018/12/05/gandiva-donation/, 2018.

[20] Introducing apache arrow flight: A framework for fast data transport. https://arrow.apache.org/blog/2019/10/13/introducing-arrow-flight/, 2019.

[21] Daniel Abadi, Peter Boncz, Stavros Harizopoulos Amiato, Stratos Idreos, and Samuel Madden. *The design and implementation of modern column-oriented database systems*. Now Hanover, Mass., 2013.

[22] Daniel J Abadi, Samuel R Madden, and Nabil Hachem. Column-stores vs. row-stores: how different are they really? In *Proceedings of the 2008 ACM SIGMOD international conference on Management of data*, pages 967–980, 2008.

[23] Martin Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. Tensorflow: A system for large-scale machine learning. In *12th {USENIX} symposium on operating systems design and implementation ({OSDI} 16)*, pages 265–283, 2016.

[24] Ian F Adams, Neha Agrawal, and Michael P Mesnier. Enabling near-data processing in distributed object storage systems. In *Proceedings of the 13th ACM Workshop on Hot Topics in Storage and File Systems*, pages 28–34, 2021.

[25] AdoptOpenJDK. Adoptopenjdk jdk mission control. `https://adoptopenjdk.net/jmc.html`. Accessed: 2021-02-02.

[26] Amazon. Amazon s3. `https://aws.amazon.com/s3/`. Accessed: 2020-11-16.

[27] Apache. Apache avro. `https://avro.apache.org/`.

[28] Apache Software Foundation. Hadoop.

[29] Michael Armbrust, Reynold S Xin, Cheng Lian, Yin Huai, Davies Liu, Joseph K Bradley, Xiangrui Meng, Tomer Kaftan, Michael J Franklin, Ali Ghodsi, et al. Spark sql: Relational data processing in spark. In *Proceedings of the 2015 ACM*

SIGMOD international conference on management of data, pages 1383–1394, 2015.

[30] Artem Babenko and Victor Lempitsky. The inverted multi-index. *IEEE transactions on pattern analysis and machine intelligence*, 37(6):1247–1260, 2014.

[31] Henri Bal, Dick Epema, Cees de Laat, Rob van Nieuwpoort, John Romein, Frank Seinstra, Cees Snoek, and Harry Wijshoff. A medium-scale distributed system for computer science research: Infrastructure for the long term. *Computer*, 49(5):54–63, 2016.

[32] Peter A Boncz, Martin L Kersten, and Stefan Manegold. Breaking the memory wall in monetdb. *Communications of the ACM*, 51(12):77–85, 2008.

[33] Goncalo Borges, Sean Crosby, and Lucien Boland. Cephfs: a new generation storage platform for australian high energy physics. In *Journal of Physics: Conference Series*, volume 898, page 062015. IOP Publishing, 2017.

[34] Dhruba Borthakur. The hadoop distributed file system: Architecture and design. *Hadoop Project Website*, 11(2007):21, 2007.

[35] Paul G Brown. Overview of scidb: large scale array storage, processing and analysis. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data*, pages 963–968, 2010.

[36] Wei Cao, Yang Liu, Zhushi Cheng, Ning Zheng, Wei Li, Wenjie Wu, Linqiang Ouyang, Peng Wang, Yijing Wang, Ray Kuan, et al. {POLARDB} meets com-

putational storage: Efficiently support analytical workloads in cloud-native relational database. In *18th {USENIX} Conference on File and Storage Technologies ({FAST} 20)*, pages 29–41, 2020.

[37] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. Apache flink: Stream and batch processing in a single engine. *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*, 36(4), 2015.

[38] Ceph. Introduction to librados. `https://docs.ceph.com/en/latest/rados/api/librados-intro/`.

[39] David Chappell et al. Introducing windows azure. *Microsoft, Inc, Tech. Rep*, 2009.

[40] Kristina Chodorow. *MongoDB: the definitive guide: powerful and scalable data storage.* " O'Reilly Media, Inc.", 2013.

[41] Eunsuk Chong, Chulwoo Han, and Frank C Park. Deep learning networks for stock market analysis and prediction: Methodology, data representations, and case studies. *Expert Systems with Applications*, 83:187–205, 2017.

[42] AWS Data Wrangler community. Aws data wrangler. `https://github.com/awslabs/aws-data-wrangler`. Accessed: 2020-11-16.

[43] Intel Corporation. Optimized analytics package for spark platform (oap). `https://github.com/Intel-bigdata/OAP`. Accessed: 2021-01-15.

[44] Eduarda Costa, Carlos Costa, and Maribel Yasmina Santos. Evaluating partitioning and bucketing strategies for hive-based big data warehousing systems. *Journal of Big Data*, 6(1):34, 2019.

[45] cuDF community. cudf - gpu dataframes. `https://github.com/rapidsai/cudf`. Accessed: 2020-11-16.

[46] Benoit Dageville, Thierry Cruanes, Marcin Zukowski, Vadim Antonov, Artin Avanes, Jon Bock, Jonathan Claybaugh, Daniel Engovatov, Martin Hentschel, Jiansheng Huang, et al. The snowflake elastic data warehouse. In *Proceedings of the 2016 International Conference on Management of Data*, pages 215–226, 2016.

[47] Debendra Das Sharma, Robert Blankenship, and Daniel Berger. An introduction to the compute express link (cxl) interconnect. *ACM Comput. Surv.*, 56(11), July 2024.

[48] Ning Ding, Yujia Qin, Guang Yang, Fuchao Wei, Zonghan Yang, Yusheng Su, Shengding Hu, Yulin Chen, Chi-Min Chan, Weize Chen, Jing Yi, Weilin Zhao, Xiaozhi Wang, Zhiyuan Liu, Hai-Tao Zheng, Jianfei Chen, Yang Liu, Jie Tang, Juanzi Li, and Maosong Sun. Delta tuning: A comprehensive study of parameter efficient methods for pre-trained language models, 2022.

[49] Jaeyoung Do, Yang-Suk Kee, Jignesh M Patel, Chanik Park, Kwanghyun Park, and David J DeWitt. Query processing on smart ssds: Opportunities and chal-

lenges. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*, pages 1221–1230, 2013.

[50] Matthew Stephen Doerner. An analysis of machine learning hardware from tenstorrent. 2024.

[51] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library, 2025.

[52] Dmitry Duplyakin, Robert Ricci, Aleksander Maricq, Gary Wong, Jonathon Duerig, Eric Eide, Leigh Stoller, Mike Hibler, David Johnson, Kirk Webb, Aditya Akella, Kuangching Wang, Glenn Ricart, Larry Landweber, Chip Elliott, Michael Zink, Emmanuel Cecchet, Snigdhaswin Kar, and Prabodh Mishra. The design and operation of CloudLab. In *Proceedings of the USENIX Annual Technical Conference (ATC)*, pages 1–14, July 2019.

[53] Manar A Elmeiligy, Ali I El Desouky, and Sally M Elghamrawy. A multidimensional big data storing system for generated covid-19 large-scale data using apache spark. *arXiv preprint arXiv:2005.05036*, 2020.

[54] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. Fast approximate nearest neighbor search with the navigating spreading-out graph. *arXiv preprint arXiv:1707.00143*, 2017.

[55] Google. Google flatbuffers. `https://google.github.io/flatbuffers/`.

[56] Anurag Gupta, Deepak Agarwal, Derek Tan, Jakub Kulesza, Rahul Pathak, Stefano Stefani, and Vidhya Srinivasan. Amazon redshift and the case for simpler data warehouses. In *Proceedings of the 2015 ACM SIGMOD international conference on management of data*, pages 1917–1923, 2015.

[57] Ezz El-Din Hemdan and DH Manjaiah. Spark-based log data analysis for reconstruction of cybercrime events in cloud environment. In *2017 International Conference on Circuit, Power and Computing Technologies (ICCPCT)*, pages 1–8. IEEE, 2017.

[58] Martin Hirzel, Henrique Andrade, Bugra Gedik, Gabriela Jacques-Silva, Rohit Khandekar, Vibhore Kumar, Mark Mendell, Howard Nasgaard, Scott Schneider, Robert Soulé, et al. Ibm streams processing language: Analyzing big data in motion. *IBM Journal of Research and Development*, 57(3/4):7–1, 2013.

[59] Qiang Huang, Jianlin Feng, Yikai Zhang, Qiong Fang, and Wilfred Ng. Query-aware locality-sensitive hashing for approximate nearest neighbor search. *Proceedings of the VLDB Endowment*, 9(1):1–12, 2015.

[60] Randall Hunt. S3 select and glacier select – retrieving subsets of objects. `https://aws.amazon.com/blogs/aws/s3-glacier-select/`, 11 2017.

[61] S Idreos, F Groffen, N Nes, S Manegold, S Mullender, and M Kersten. Monetdb: Two decades of research in column-oriented database. *IEEE Data Engineering Bulletin*, 2012.

[62] Intel. Daos client apis, tools and i/o middleware. https://github.com/daos-stack/daos/blob/master/src/README.md#client-apis-tools-and-io-middleware.

[63] Muhammad Hussain Iqbal and Tariq Rahim Soomro. Big data analysis: Apache storm perspective. *International journal of computer trends and technology*, 19(1):9–14, 2015.

[64] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. Diskann: Fast accurate billion-point nearest neighbor search on a single node. *Advances in neural information processing Systems*, 32, 2019.

[65] Insoon Jo, Duck-Ho Bae, Andre S Yoon, Jeong-Uk Kang, Sangyeun Cho, Daniel DG Lee, and Jaeheon Jeong. Yoursql: a high-performance database system leveraging in-storage computing. *Proceedings of the VLDB Endowment*, 9(12):924–935, 2016.

[66] Hyungyo Kim, Gaohan Ye, Nachuan Wang, Amir Yazdanbakhsh, and Nam Sung Kim. Exploiting intel advanced matrix extensions (amx) for large language model inference. *IEEE Computer Architecture Letters*, 23(1):117–120, 2024.

[67] Ross Lawley. Apache spark connector for mongodb. https://www.slideshare.net/mongodb/how-to-connect-spark-to-your-own-datasource, https:

//databricks.com/blog/2015/03/20/using-mongodb-with-spark.html. Accessed: 2020-11-06.

[68] Jeff LeFevre. Skyhook data management. https://skyhookdm.com/, August 2020.

[69] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.

[70] Zhen Liang, Johann Lombardi, Mohamad Chaarawi, and Michael Hennecke. Daos: A scale-out high performance storage stack for storage class memory. In *Asian Conference on Supercomputing Frontiers*, pages 40–54. Springer, 2020.

[71] Yu A Malkov and Dmitry A Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence*, 42(4):824–836, 2018.

[72] MATLAB. *version 7.10.0 (R2010a)*. The MathWorks Inc., Natick, Massachusetts, 2010.

[73] Rafael Moreno-Vozmediano, Rubén S. Montero, and Ignacio M. Llorente. Iaas cloud architecture: From virtualized datacenters to federated cloud infrastructures. *Computer*, 45(12):65–72, 2012.

[74] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard

Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging ai applications. In *Proceedings of the 13th USENIX Conference on Operating Systems Design and Implementation*, OSDI'18, page 561–577, USA, 2018. USENIX Association.

[75] Zeyad Assi Obaid Nasri Sulaiman, MH Marhaban, and MN Hamidon. Design and implementation of fpga-based systems-a review. *Australian Journal of Basic and Applied Sciences*, 3(4):3575–3596, 2009.

[76] Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G. Azzolini, Dmytro Dzhulgakov, Andrey Mallevich, Ilia Cherniavskii, Yinghai Lu, Raghuraman Krishnamoorthi, Ansha Yu, Volodymyr Kondratenko, Stephanie Pereira, Xianjie Chen, Wenlin Chen, Vijay Rao, Bill Jia, Liang Xiong, and Misha Smelyanskiy. Deep learning recommendation model for personalization and recommendation systems, 2019.

[77] Martin Odersky. The scala programming language. `https://www.scala-lang.org/`. Accessed: 2020-11-08.

[78] OpenStack. Openstack swift storlet engine overview. `https://docs.openstack.org/storlets/latest/`.

[79] OpenStack. Storlet's documentation. `https://docs.openstack.org/storlets/latest/`.

[80] Oracle. Oracle exadata. https://www.oracle.com/engineered-systems/exadata/.

[81] Weston Pace, Chang She, Lei Xu, Will Jones, Albert Lockett, Jun Wang, and Raunak Shah. Lance: Efficient random access in columnar storage through adaptive structural encodings. *arXiv preprint arXiv:2504.15247*, 2025.

[82] Sung Joo Park, H Kim, KS Kim, J So, J Ahn, WJ Lee, D Kim, Young-Ju Kim, J Seok, JG Lee, et al. Scaling of memory performance and capacity with cxl memory expander. In *HCS*, pages 1–27, 2022.

[83] Pedro Pedreira, Orri Erling, Masha Basmanova, Kevin Wilfong, Laith Sakka, Krishna Pai, Wei He, and Biswapesh Chattopadhyay. Velox: meta's unified execution engine. *Proceedings of the VLDB Endowment*, 15(12):3372–3384, 2022.

[84] Gregory F Pfister. An introduction to the infiniband architecture. *High performance mass storage and parallel I/O*, 42(617-632):10, 2001.

[85] Raghu Prabhakar, Ram Sivaramakrishnan, Darshan Gandhi, Yun Du, Mingran Wang, Xiangyu Song, Kejie Zhang, Tianren Gao, Angela Wang, Xiaoyan Li, Yongning Sheng, Joshua Brot, Denis Sokolov, Apurv Vivek, Calvin Leung, Arjun Sabnis, Jiayu Bai, Tuowen Zhao, Mark Gottscho, David Jackson, Mark Luttrell, Manish K. Shah, Zhengyu Chen, Kaizhao Liang, Swayambhoo Jain, Urmish Thakker, Dawei Huang, Sumti Jairath, Kevin J. Brown, and Kunle Olukotun. Sambanova sn40l: Scaling the ai memory wall with dataflow and composition of

experts. In *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1353–1366, 2024.

[86] Mark Raasveldt and Hannes Mühleisen. Data management for data science-towards embedded analytics. In *CIDR*, 2020.

[87] Federico Razzoli. *Mastering MariaDB*. Packt Publishing Ltd, 2014.

[88] Matthew Rocklin. Dask: Parallel computation with blocked algorithms and task scheduling. In *Proceedings of the 14th python in science conference*, volume 126. Citeseer, 2015.

[89] Robert B Ross, George Amvrosiadis, Philip Carns, Charles D Cranor, Matthieu Dorier, Kevin Harms, Greg Ganger, Garth Gibson, Samuel K Gutierrez, Robert Latham, et al. Mochi: Composing data services for high-performance computing environments. *Journal of Computer Science and Technology*, 35(1):121–144, 2020.

[90] Gabin Schieffer, Jacob Wahlgren, Jie Ren, Jennifer Faj, and Ivy Peng. Harnessing integrated cpu-gpu system memory for hpc: a first look into grace hopper. In *Proceedings of the 53rd International Conference on Parallel Processing*, pages 199–209, 2024.

[91] Naman Shah. Determining disease outbreak influence from voluminous epidemiology data on enhanced distributed graph-parallel system. *2000-2019-CSU Theses and Dissertations*, 2017.

[92] R. Shree, T. Choudhury, S. C. Gupta, and P. Kumar. Kafka: The modern platform

for data management and analysis in big data domain. In *2017 2nd International Conference on Telecommunication and Networks (TEL-NET)*, pages 1–5, 2017.

[93] Konstantin Shvachko, Hairong Kuang, Sanjay Radia, and Robert Chansler. The hadoop distributed file system. In *2010 IEEE 26th symposium on mass storage systems and technologies (MSST)*, pages 1–10. Ieee, 2010.

[94] Malcolm Singh and Ben Leonhardi. Introduction to the ibm netezza warehouse appliance. In *Proceedings of the 2011 Conference of the Center for Advanced Studies on Collaborative Research*, pages 385–386, 2011.

[95] Mark Slee, Aditya Agarwal, and Marc Kwiatkowski. Thrift: Scalable cross-language services implementation. *Facebook white paper*, 5(8):127, 2007.

[96] Statista. Number of mobile phone users worldwide from 2015 to 2020 (in billions). `https://www.statista.com/statistics/871513/worldwide-data-created/`. Accessed:2020-10-08.

[97] Arrow Development Team. Apache arrow. `https://arrow.apache.org`, 10 2018.

[98] Animesh Trivedi, Patrick Stuedi, Jonas Pfefferle, Adrian Schuepbach, and Bernard Metzler. Albis: High-performance file format for big data systems. In *2018 {USENIX} Annual Technical Conference ({USENIX}{ATC} 18)*, pages 615–630, 2018.

[99] Animesh Trivedi, Patrick Stuedi, Jonas Pfefferle, Adrian Schuepbach, and Bernard

Metzler. Albis: High-performance file format for big data systems. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pages 615–630, 2018.

[100] The univocity team. univocity-parsers. `https://github.com/uniVocity/univocity-parsers`. Accessed: 2021-02-01.

[101] Guido Van Rossum et al. Python, 1991.

[102] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, et al. Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 international conference on management of data*, pages 2614–2627, 2021.

[103] Xingwei Wang, Hong Zhao, and Jiakeng Zhu. Grpc: A communication cooperation mechanism in distributed systems. *ACM SIGOPS Operating Systems Review*, 27(3):75–86, 1993.

[104] Ying Wei, Yi Chieh Huang, Haiming Tang, Nithya Sankaran, Ish Chadha, Dai Dai, Olakanmi Oluwole, Vishnu Balan, and Edward Lee. 9.3 nvlink-c2c: A coherent off package chip-to-chip interconnect with 40gbps/pin single-ended signaling. In *2023 IEEE International Solid-State Circuits Conference (ISSCC)*, pages 160–162, 2023.

[105] Sage A. Weil, Scott A. Brandt, Ethan L. Miller, Darrell D. E. Long, and Carlos Maltzahn. Ceph: A scalable, high-performance distributed file system. In *Pro-*

ceedings of the 7th Symposium on Operating Systems Design and Implementation, OSDI '06, page 307–320, USA, 2006. USENIX Association.

[106] Sage A Weil, Scott A Brandt, Ethan L Miller, Darrell DE Long, and Carlos Maltzahn. Ceph: A scalable, high-performance distributed file system. In *Proceedings of the 7th symposium on Operating systems design and implementation*, pages 307–320, 2006.

[107] Sage A Weil, Scott A Brandt, Ethan L Miller, and Carlos Maltzahn. Crush: Controlled, scalable, decentralized placement of replicated data. In *SC'06: Proceedings of the 2006 ACM/IEEE Conference on Supercomputing*, pages 31–31. IEEE, 2006.

[108] Sage A Weil, Andrew W Leung, Scott A Brandt, and Carlos Maltzahn. Rados: a scalable, reliable storage service for petabyte-scale storage clusters. In *Proceedings of the 2nd international workshop on Petascale data storage: held in conjunction with Supercomputing'07*, pages 35–44, 2007.

[109] Tom White. *Hadoop: The definitive guide.* " O'Reilly Media, Inc.", 2012.

[110] Hadley Wickham. Feather: A fast on-disk format for data frames for r and python, powered by apache arrow. `https://www.rstudio.com/blog/feather/`.

[111] Wikipedia. Ceph (software). `https://bit.ly/3mkapHU`.

[112] Louis Woods, Zsolt István, and Gustavo Alonso. Ibex: An intelligent storage

engine with support for advanced sql offloading. *Proceedings of the VLDB Endowment*, 7(11):963–974, 2014.

[113] Po-Yen Wu, Chih-Wen Cheng, Chanchala D Kaddi, Janani Venugopalan, Ryan Hoffman, and May D Wang. –omic and electronic health record big data analytics for precision medicine. *IEEE Transactions on Biomedical Engineering*, 64(2):263–273, 2016.

[114] Reynold S Xin, Joseph E Gonzalez, Michael J Franklin, and Ion Stoica. Graphx: A resilient distributed graph system on spark. In *First international workshop on graph data management experiences and systems*, pages 1–6, 2013.

[115] Qiumin Xu, Huzefa Siyamwala, Mrinmoy Ghosh, Tameesh Suri, Manu Awasthi, Zvika Guz, Anahita Shayesteh, and Vijay Balakrishnan. Performance analysis of nvme ssds and their implication on real world databases. In *Proceedings of the 8th ACM International Systems and Storage Conference*, pages 1–11, 2015.

[116] Xiangyao Yu, Matt Youill, Matthew Woicik, Abdurrahman Ghanem, Marco Serafini, Ashraf Aboulnaga, and Michael Stonebraker. Pushdowndb: Accelerating a dbms using s3 computation. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*, pages 1802–1805. IEEE, 2020.

[117] Matei Zaharia, Mosharaf Chowdhury, Michael J Franklin, Scott Shenker, Ion Stoica, et al. Spark: Cluster computing with working sets. *HotCloud*, 10(10-10):95, 2010.

[118] Matei Zaharia, Mosharaf Chowdhury, Michael J Franklin, Scott Shenker, Ion Stoica, et al. Spark: Cluster computing with working sets. *HotCloud*, 10(10-10):95, 2010.

[119] Matei Zaharia, Reynold Xin, Xiao Li, Fan Wenchen, and Yin Huai. Introducing apache spark 3.0. `https://databricks.com/blog/2020/06/18/introducing-apache-spark-3-0-now-available-in-databricks-runtime-7-0.html`. Accessed:2020-10-16.

[120] Hongze Zhang. Arrow-7808: [java][dataset] implement dataset java api by jni to c++. `https://github.com/zhztheplayer/arrow-1/tree/ARROW-7808`. Accessed: 2020-09-14.